Gisma: Giant-Step-Small-Step Indexing for Approximate Similarity Search in Graph Databases

Yikai Gong
Hong Kong Baptist University
ykgong@comp.hkbu.edu.hk

Lyu Xu
Nanyang Technological University
lyu.xu@ntu.edu.sg

Yun Peng
Guangzhou University
yunpeng@gzhu.edu.cn

Byron Choi
Hong Kong Baptist University
choi@hkbu.edu.hk

Jianliang Xu
Hong Kong Baptist University
xujl@comp.hkbu.edu.hk

Xin Huang
Hong Kong Baptist University
xinhuang@comp.hkbu.edu.hk

ABSTRACT

Approximate similarity search in graph databases retrieves graphs whose *graph edit distance* (GED) to a given query graph is within a given threshold. This paper first shows that the GED space is *doubling*, as any ball of radius r can be covered by a bounded number of balls of radius $r/2$. Despite being a fundamental property for indexing metric spaces, it has not yet been investigated in the context of approximate similarity search in graphs. In particular, when compared to other metric spaces, distance computations (as GED is *NP-hard*) in query processing are minimized. Next, this paper reveals a *two-stage phenomenon* in the GED space: when GEDs exceed a threshold α, the search on doubling-based indexes can efficiently identify the subspaces containing answer graphs; however, when GEDs are smaller than α, doubling-based indexes are inefficient. Based on these insights, we propose **Gi**ant-Step-**Sma**ll-Step (Gisma), a novel two-stage index for approximate graph similarity search. The first stage of Gisma, namely NetDAG, is a doubling-based index, in which we analyze the range of child balls of an index node to facilitate a coarse-grained search for answer subspaces achieving the *lowest known query time complexity*. In Gisma's second stage, EPF indexes graph edit paths in each subspace, achieving an efficient fine-grained search for answer graphs. Experiments on four benchmark datasets demonstrate that Gisma outperforms the state-of-the-art in both efficiency and effectiveness.

PVLDB Reference Format:
Yikai Gong, Lyu Xu, Yun Peng, Byron Choi, Jianliang Xu, and Xin Huang. Gisma: Giant-Step-Small-Step Indexing for Approximate Similarity Search in Graph Databases. PVLDB, 20(1): XXX-YYY, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/McKayGONG/Gisma.

1 INTRODUCTION

Similarity search in graph databases aims to retrieve graphs that are similar to a given query graph. It is a fundamental problem with a wide range of applications, *e.g.,* [1, 2, 7, 15, 18, 40, 44, 48–50, 56, 58]. For example, in bioinformatics, production ligand discovery services such as SmallWorld [48] search molecule databases by graph edit distance, which is also used to measure the quality of nearest-neighbour search in compound libraries [35].

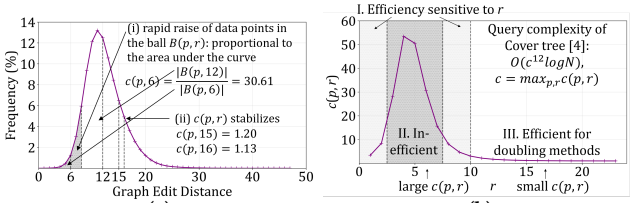


Figure 1: (a) Distribution of pairwise GEDs in the AIDS dataset (graphs with at most 15 vertices). The area under the curve up to r is proportional to $|B(p,r)|$, and the ratio $|B(p,2r)|/|B(p,r)|$ is $c(p,r)$; (b) $c(p,r)$ under the same setting. Three cases of doubling methods' efficiency: I. Efficiency sensitive to r, II. Inefficient, and III. Efficient.

For graph similarity search, the *graph edit distance* (GED), which is the minimum number of edit operations to transform a graph into another, has been widely used as the similarity *metric* in the literature [3, 8, 9, 32, 33, 42, 44, 53]. Specifically, given a query graph Q and a distance threshold τ, the GED similarity search is to retrieve all graphs from the graph database whose GED to Q is within τ. As GED computation/verification is NP-hard [54], the GED similarity search over large graph databases is computationally expensive, and hence, a variety of approximate methods have been proposed to efficiently answer graph similarity search queries in recent years.

Existing solutions. In general, the main ideas of existing works on approximate graph similarity search fall into two categories: (i) to reduce the number of GED computations, and (ii) to accelerate individual GED computations. Especially, for category (i), GHashing [44] uses graph neural networks with hashing for efficient filtering, while LAN [42] is a proximity graph-based approximate k-nearest neighbor search method on graph databases; for category (ii), App-BMao [51] extends the state-of-the-art exact GED verification method AStar-BMao [9] by using an iteration limit to compute approximate GED, achieving speedups with a slight accuracy loss, whereas GEDHOT [10] adopts an ensemble strategy that combines supervised and unsupervised models to compute approximate GED.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

There have also been extensive works that study the dimensionality of the metric space for building similarity search indexes [6, 11, 12, 22, 25, 31, 36], among which the doubling metrics measure the dimensionality through a doubling constant λ [36] and an expansion constant c [31]. These works emphasize dimensionality analysis and time complexity sublinear in the database size N (including recent works [13, 19, 28] under bounded doubling dimension), while their empirical evaluation is limited (*e.g.,* Cover Tree [6]). In contrast, existing methods for graphs [10, 32, 42, 44, 51] provide no such time-complexity guarantees. *This work is the first to investigate doubling metrics in the context of graph similarity search queries.*

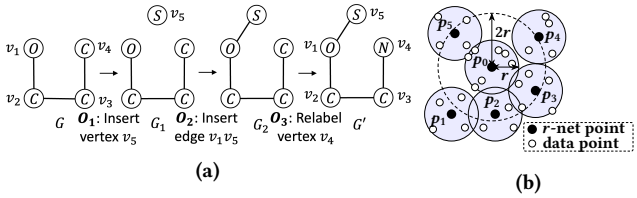
Doubling metrics. Given a set $\mathcal{X}$ of data points and a distance metric d, the metric space $(\mathcal{X}, d)$ is *doubling* if it has a finite *doubling constant* λ, defined as the smallest number such that: for every data point $p \in \mathcal{X}$ and every distance $r > 0$, the ball $B(p, 2r)$ can *always* be covered by at most λ balls of radius r, where a ball $B(p, r) = \{q \in \mathcal{X} \mid d(p, q) \leq r\}$. For ease of presentation, we call such a metric space a *doubling space*, and denote by $\lambda(p, r)$ the minimum number of balls of radius r to cover $B(p, 2r)$. As another measure of dimensionality, the *expansion constant* c is defined as the maximum ratio $|B(p, 2r)|/|B(p, r)|$ over all p and r. We also denote the ratio $|B(p, 2r)|/|B(p, r)|$ for given p and r by $c(p, r)$.

Challenges. There are several challenges in introducing methods for doubling metrics. First, indexes built on doubling metrics have a query time complexity (*e.g.,* $O(\lambda^{O(1)} \log N)$ for Net-tree [25] and $O(c^{12} \log N)$ for Cover Tree [6]). It should be remarked that the indexes are efficient *only when* doubling/expansion constants (λ or c) are small, *e.g.,* $\lambda^{O(1)}$ or c^{12}. Second, an application of existing doubling indexes to the GED space is technically intriguing because each GED computation is costly, when compared to those of other distance computations. In the query traversal of those indexes, the algorithms proceed to *a child candidate set*. In particular, for Net-Tree [25], the per-layer *number of distance computations* (NDC) is $\lambda_\alpha^{O(1)}$ with a large exponent, *i.e.,* $\lambda_\alpha^{4+\log_2 52}$. Therefore, NDC and the distance computation must be minimized.

Empirical observations. As mentioned above, the doubling constant λ or the expansion constant c are crucial to the query time complexity of index-based search methods. We first conduct a preliminary evaluation of the distribution of GED and the curves of $\lambda(p, r)$ and $c(p, r)$.

Fig. 1(a) shows the distribution of exact GED between graphs in the AIDS dataset with at most 15 vertices, while Fig. 1(b) shows the curve of $c(p, r)$ under the same setting. We omit the numerical results for $\lambda(p, r)$ since λ and c are closely related with $\lambda \leq c^4$ [36]. Then, our observations are as follows:

- Fig. 1(a) reveals a *unimodal* distribution of GEDs: the frequency rises rapidly, reaches a peak, and then decays gradually into a long tail. This pattern is consistent across different databases, suggesting an intrinsic property of the GED space.[1]
- Consider viewing Fig. 1(a) from a single query graph's perspective: fixing a graph p, the curve reflects the distribution of GEDs from p to all other graphs [27]. From this perspective, the (grey) area under the curve up to radius r corresponds to $|B(p, r)|$, and then the ratio of the areas at $2r$ and r corresponds to $c(p, r) = |B(p, 2r)|/|B(p, r)|$. Such a distribution curve reveals a two-stage phenomenon of the GED space: *when r is small, $|B(p, 2r)|$ is much larger than $|B(p, r)|$, making $c(p, r)$ large; when r is large, $c(p, r)$ remains small and stable.* Fig. 1(b) further illustrates this phenomenon by showing $c(p, r)$ against r, where three cases can be identified: Cases I and II are in a small-GED stage, where large or unstable $c(p, r)$ makes doubling methods inefficient; Case III is in a large-GED stage, where $c(p, r)$ is small and stable and doubling methods become efficient. As the expansion constant $c = \max_{p,r} c(p, r)$ is dominated by the large values at small distances, it is inefficient to directly apply existing doubling metrics-based indexes. Such a two-stage phenomenon can also be observed in other datasets (see Sec. 3), revealing that no single doubling-based index remains efficient across the entire GED space.



(a) (b)

Figure 2: (a) A graph edit path $[O_1, O_2, O_3]$ that transforms graph G into graph G', resulting in a GED of 3; (b) An illustration of an r-net, where solid dots p_i, $0 \leq i \leq 5$ are r-net centers, and small circles are data points in database $\mathcal{D}$.

Overview of our solution. Building on the above insights, we propose a two-stage index called Gisma to achieve efficient query processing for approximate graph similarity search. The first stage, NetDAG, uses doubling metrics to locate the graphs when GED is larger than a radius threshold α, where the *doubling constant down to* α is denoted as $\lambda_\alpha = \max_{p, r \geq \alpha} \lambda(p, r)$. Specifically, NetDAG organizes the graph database into a hierarchical structure, where each ball B can be covered by B's child-layer balls having a radius equal to half of B's radius, which is in line with the doubling constant λ. This allows traversal by giant steps, efficiently pruning the entire search space to small subspaces (*i.e.,* balls in NetDAG). The second stage, namely EditPathForest (EPF), is built for GEDs smaller than α. EPF is a set of EditPathTrees (EPTs), each of which is a tree-like index constructed from the graph edit paths whose lengths equal GEDs between data graphs, and is used for *small-step* search within balls of radius α. Gisma is consistent with an observation on proximity graph-based indexes for ANNS search on non-graph data [17, 24, 30, 38, 52]: the search quickly approaches the query from a distant starting point, but converges slowly near it.

Contributions. The contributions of this paper are as follows:

- We analyze the GED space and find that its doubling/expansion constant is small at large distances but large at small distances, exhibiting a two-stage phenomenon. Accordingly, we propose a novel two-stage index called Gisma to achieve efficient approximate graph similarity search.
- For the first stage of Gisma, we propose NetDAG, a hierarchical index that supports *giant-step* traversals and achieves $O(\lambda_\alpha^{\log_2 20} \cdot \phi)$ worst-case query time (where λ_α is the doubling constant down to the radius threshold α and ϕ is the number of layers in NetDAG),

[1] The distribution is consistent with that of approximate GED computed by GREED [45] on the entire AIDS dataset.

which is the lowest among those of existing doubling-based methods. This is achieved by maintaining a single candidate pivot instead of a set during the traversal of the index. Gisma uses NetDAG only for the case of GED $\geq \alpha$, to avoid the high computational complexity at small GEDs due to a large doubling constant.

- For Gisma's second stage, we propose EditPathForest, which indexes the graph edit paths used to compute the GED between graphs, which exploits the property that graphs with small GED share common edit-path prefixes, which enables *small-step* searching within balls of radius α, together with three optimizations.
- We conduct comprehensive experiments on four widely-used benchmark datasets against six representative methods. The results show that Gisma outperforms these methods in almost all cases, and the speedup is more significant when τ is large, *e.g.*, reducing query time by up to 35.9% over App-BMao+ [51].

Organization. The rest of this paper is organized as follows. Sec. 2 introduces the preliminaries, background, and Gisma's overview. Sec. 3 analyzes the GED space. Secs. 4 and 5 detail the first and the second stage of Gisma, respectively, while Sec. 6 presents a cost model for Gisma. Sec. 7 presents comprehensive experiments, and Sec. 8 reviews related work. Finally, Sec. 9 concludes the paper. Due to space limitations, proofs and additional experiments are presented in the appendices of the technical report [20].

2 PRELIMINARIES, PROBLEM STATEMENT AND SOLUTION OVERVIEW

In this section, we introduce the preliminaries of approximate similarity search in graph databases. Then, we present some background and an overview of our index Gisma. For clarity, we summarize the frequently used notations in Tab. 1.

2.1 Preliminaries

We study a database of undirected graphs with vertex labels but without edge labels, following prior work [10, 42–44]. This setting is compatible with existing graph similarity search methods that incorporate edge labels [9, 32, 51].

Graph. A graph is denoted by $G = (V, E, \ell)$, where V and E are the sets of vertices and undirected edges of G, respectively. The mapping $\ell : V \rightarrow L$ is a function that maps a vertex in V to a label. For clarity, we use $V(G)$ and $E(G)$ to denote the vertex set and edge set of a graph G, respectively.

Graph edit path and distance [10, 42–44, 53]. We consider five edit operations on graphs, including vertex insertion, vertex deletion, vertex relabeling, edge insertion, and edge deletion. Following prior work [10, 42–44], we adopt the unit-cost edit model, in which each edit operation has unit cost.[2] Given two graphs G and G', a *graph edit path* (GEP) from G to G' is the *shortest* sequence of edit operations on G that transforms G into G'; and the *graph edit distance* (GED) between G and G', denoted by $d(G, G')$, is the length of the GEP from G to G'.

Example 2.1. Fig. 2(a) shows a graph edit path from G to G' with the sequence $[O_1, O_2, O_3]$ of edit operations, where (i) O_1 is the vertex insertion of vertex v_5 with label S; (ii) O_2 is the edge insertion of edge (v_1, v_5); (iii) O_3 is vertex relabeling that changes v_4's label from C to N. As $[O_1, O_2, O_3]$ has the minimum length of 3, the graph edit distance between G and G' is $d(G, G') = 3$.

[2]Weighted or attribute-dependent cost models do not generally lead to metric spaces.

Table 1: Frequently used notations

Notation	Description
$\mathcal{D}; Q; G$	A database of graphs; a query graph Q; a (data) graph G in $\mathcal{D}$
$d(G, G')$	The graph edit distance (GED) between two graphs G, G'
τ	The GED threshold for graph similarity search
$q(Q, \tau)$	A similarity search query with query graph Q and range τ
$(\mathcal{X}, d)$	A metric space where $\mathcal{X}$ is a set of points and d is the GED
$B(p, r)$	$\{p' \in \mathcal{X} \mid d(p, p') \leq r\}$, a set of points within distance r from p
r-ball	A ball $B(p, r)$ for some center p
$\lambda; c$	Doubling constant; expansion constant
$\lambda(p, r)$; $c(p, r)$	Min. number of r-balls to cover $B(p,2r)$; $\lvert B(p,2r)\rvert / \lvert B(p,r)\rvert$
α	The radius threshold parameter of Gisma
$\lambda_\alpha; c_\alpha$	Doubling/expansion constant down to α
Y_i	The i-th layer of NetDAG, an $(\alpha \cdot 2^i)$-net of $\mathcal{D}$
ϕ	Number of layers in NetDAG
p_j^i	The point p_j in layer Y_i of NetDAG

Computing the exact GED is NP-hard [54], which makes graph similarity search computationally costly on large databases. Therefore, in this paper, we focus on approximate graph similarity search, and propose an index-based solution to tackle the search.

Problem statement. Given a graph database $\mathcal{D} = \{G_1, \ldots, G_N\}$ and a query $q(Q, \tau)$ specified by a query graph Q and a threshold τ, the *approximate graph similarity search* problem is to retrieve approximate answer graphs in $\mathcal{D}$, where the answer graphs G satisfy that $d(G, Q)$ (GED between G and Q) is at most τ. This paper aims to provide an efficient solution with a high recall.

2.2 Background of Doubling Metric Spaces

For ease of Gisma's illustration, we introduce some fundamental concepts in doubling metric spaces, including the doubling constant λ [36], the expansion constant c [6, 31], doubling spaces [36], and r-nets [25, 36].

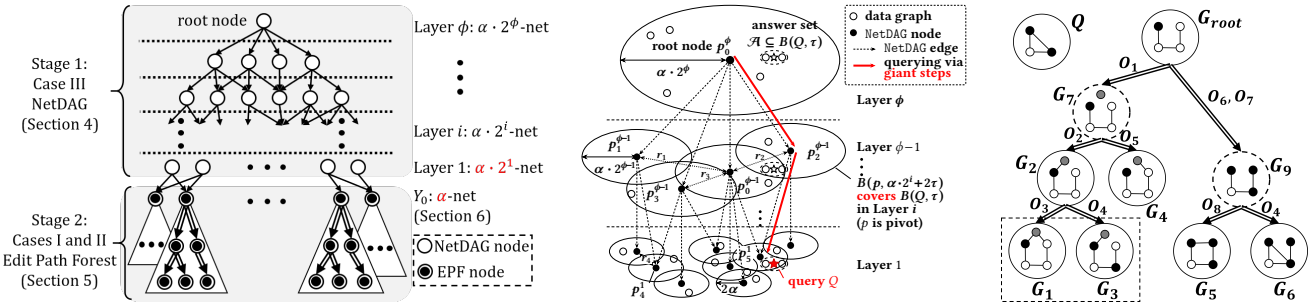
Definition 2.1 (*Doubling constant* [36])**.** Given a metric space $(\mathcal{X}, d)$, the *doubling constant* λ is the smallest number such that every ball $B(p, 2r)$ can be covered by at most λ balls of radius r, for all $p \in \mathcal{X}$ and $r > 0$, where $\mathcal{X}$ is the set of data points and d is a distance metric.

Remark. For a given p and r, we denote by $\lambda(p, r)$ the minimum number of r-balls (*i.e.,* balls of radius r) needed to cover $B(p, 2r)$. Then, we have $\lambda = \max_{p \in \mathcal{X}, r>0} \lambda(p, r)$. We denote by $\lambda_\alpha := \max_{p \in \mathcal{X}, r \geq \alpha} \lambda(p, r)$ the *doubling constant down to a radius threshold* α.

Definition 2.2 (*Expansion constant* [6, 31])**.** Given a metric space $(\mathcal{X}, d)$, the *expansion constant* c is the smallest value $c \geq 2$ such that $|B(p, 2r)| \leq c \cdot |B(p, r)|$ for all $p \in \mathcal{X}$ and $r > 0$.

Remark. For a given p and r, we denote $c(p, r) = |B(p, 2r)|/|B(p, r)|$. Then $c = \max_{p \in \mathcal{X}, r>0} c(p, r)$. We denote by $c_\alpha := \max_{p \in \mathcal{X}, r \geq \alpha} c(p, r)$ the *expansion constant down to* the radius threshold α.

Definition 2.3 (*Doubling space* [25, 36])**.** A metric space $(\mathcal{X}, d)$ is *doubling* if its doubling constant λ is finite.



(a) Overview of Gisma **(Stage 1:** NetDAG**; Stage 2:** EditPathForest**)** **(b) An example of** NetDAG **(c) An example of** EditPathTree

Figure 3: Overview of Gisma **– Stage 1:** NetDAG **(Sec. 4); and Stage 2:** EditPathForest **(Sec. 5). Both indices depend on the threshold α: the leaf layer of** NetDAG **is a 2α-net, and α is the radius of the balls indexed by** EPTs**.**

Example 2.2 (Computing $\lambda(p, r)$ and $c(p, r)$). In Fig. 2(b), for p_0 and the radius r shown in the figure, $B(p_0, 2r)$ can be covered by six r-balls, so $\lambda(p_0, r) = 6$. Moreover, $|B(p_0, r)| = 8$ and $|B(p_0, 2r)| = 19$. Hence, $c(p_0, r) = 19/8 = 2.38$.

Definition 2.4. (r-*net* [36]) Given a metric space $(\mathcal{X}, d)$ and a radius r, where $\mathcal{X}$ is a set of points and d is the distance measure, an r-*net* $Y \subseteq \mathcal{X}$ is a subset of $\mathcal{X}$ that satisfies the following:

- For $p_i, p_j \in Y$, $p_i \neq p_j \Rightarrow d(p_i, p_j) \geq r$; and
- $\mathcal{X} = \bigcup_{p \in Y} B(p, r)$, where $B(p, r) = \{p' \in \mathcal{X} \mid d(p, p') \leq r\}$.

Intuitively, the first condition ensures that the points in Y are r-*separated* [25] (*i.e.,* pairwise distance $\geq r$), and the second condition ensures that the collection of r-balls $\{B(p, r) \mid p \in Y\}$ covers the entire space $\mathcal{X}$.

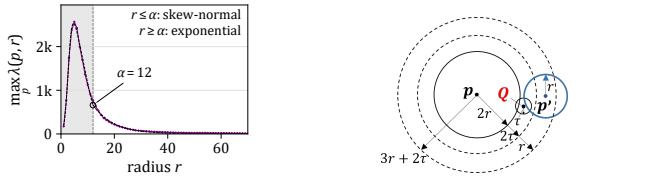
Example 2.3. Fig. 2(b) shows the space $\mathcal{X}$, where each node is a data point in $\mathcal{X}$. $Y = \{p_0, p_1, \ldots, p_5\}$ forms an r-net of $\mathcal{X}$. The balls of radius r centered at p_i for $i \in \{0, \ldots, 5\}$ together cover all points, and the centers satisfy $d(p_i, p_j) \geq r$ for any $i \neq j$.

2.3 Gisma Overview

We have now presented sufficient background to give an overview of Gisma, shown in Fig. 3(a). At the top of Fig. 3(a), Gisma's first stage index (NetDAG) consists of multiple layers of nodes satisfying that (i) each layer is an r-net (Def. 2.4) with a different radius r, (ii) for adjacent layers, the child layer's radius is half of the parent layer's radius, and (iii) there exists an edge from a parent node p to a child node p' if the GED between p and p' is within a certain distance (detailed in Sec. 4). The resulting structure is a DAG. We denote NetDAG's 2α-net Y_1 as its leaf layer.

Regarding the second-stage index, EditPathForest (EPF), it consists of a set of EditPathTrees (EPTs) such that (i) their roots are data graphs in $\mathcal{D}$, forming an α-net Y_0, where α is the radius threshold for separating Gisma's two stages (Sec. 6); and (ii) each EPT indexes the graph edit paths from its root p to the graphs in $B(p, \alpha)$. Each point in Y_1 has edges to points in Y_0 within a certain distance (same rule as between layers in NetDAG), bridging the two stages.

To answer graph similarity search queries, we first traverse NetDAG top-down, selecting at each layer a single child of the current pivot close to Q. Upon reaching the leaf layer, we collect the α-balls of the pivot's children in Y_0 and perform a depth-first search on their EPTs to retrieve the query answers.



(a) Two-stage fitting of AIDS **(b) Traversal from pivot to child**

Figure 4: (a) A two-stage fitting of $\max_p \lambda(p, r)$ on AIDS**, which splits at α into (i) a peak ($R^2 = 0.99$, $p < 0.001$) and (ii) an exponential tail ($R^2 = 0.98$, $p < 0.001$). (b) Traversal from pivot p ($d(p, Q) \leq 2r + \tau$) to child $p' \in B(p, 3r + 2\tau)$ with $d(p', Q) \leq r + \tau$, where $r = \alpha \cdot 2^i$.**

3 ANALYSIS OF GED SPACE

In this section, we show that the GED space is a doubling space, and analyze the corresponding doubling constant.

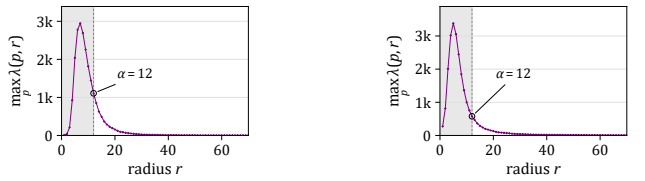
Lemma 3.1. *Given the set $\mathcal{X}$ of graphs with at most n vertices and at most ρ distinct labels, where n and ρ are constants, the metric space $(\mathcal{X}, \text{GED})$ induced by the graph edit distance is a doubling space.*

With n and ρ fixed, the number of possible graphs is finite; let M denote this number. For any ball $B(p, 2r)$ in $(\mathcal{X}, \text{GED})$, it can be covered by at most $|B(p, 2r)|$ balls of radius r, each centered at a graph in $B(p, 2r)$. Since the number of graphs in $B(p, 2r)$ never exceeds M, the doubling constant is at most M. Hence, $(\mathcal{X}, \text{GED})$ is a doubling space.

Since the query complexity of doubling-based indexes [25, 36] depends on λ (Sec. 1), their efficiency highly depends on the value of λ, which is closely related to the fanout of each node in the index structure. Lemma 3.1 shows that the GED space is doubling, which enables doubling-based indexes. We then analyze $\lambda(p, r)$ below.

Net-tree [25] proposed a method to estimate the global doubling constant λ. We extend it in Gisma to provide $\max_p \lambda(p, r)$ for each radius r, revealing how the doubling behavior varies with r. Due to the high computational cost of exact GED, we use GREED [45] to approximate GED in this process.

The doubling constant is the peak value of the $\max_p \lambda(p, r)$ curve, which equals to $\max_{p,r} \lambda(p, r)$; on AIDS, it exhibits a sharp peak and a long tail (Fig. 4(a)). The shape of the curve has significant consequences for indexing. (i) When r is small, $\max_p \lambda(p, r)$ has a peak, meaning that a $2r$-ball requires many r-balls to cover. Hence, a doubling-based index has a large fanout at r and is inefficient.



(a) PubChem **(b)** Chemical1M

Figure 5: $\max_p \lambda(p, r)$ **versus radius** r **on** PubChem **and** Chemical1M**. Both show the same two-stage shape as** AIDS **(Fig. 4(a)): a sharp peak at small** r **and a long tail at large** r**.**

(ii) When r is large, the curve has a long tail, where $\max_p \lambda(p, r)$ remains small, meaning that a $2r$-ball can be covered by only a few r-balls. Hence, the index has a small fanout at r and supports efficient traversal. We further define a doubling constant down to α as $\lambda_\alpha = \max_{p,\, r \geq \alpha} \lambda(p, r)$, which characterizes large rs. This is why Gisma applies its doubling-based index NetDAG only (Sec. 4) to the large-GED stage (GED $\geq \alpha$) and delegates the small-GED stage to EPF (Sec. 5).

These shapes can be found in other datasets (*e.g.,* Fig. 5 of PubChem and Chemical1M). Fig. 12 (Appx. A.4 [20]) plots three more datasets spanning the molecular and social-network domains (IMDB, Mutagenicity, and CANCER). The details are presented in Appx. A.4 [20].

4 NetDAG: HIERARCHICAL DOUBLING-BASED INDEX FOR GRAPHS HAVING LARGE GEDS

In this section, we propose NetDAG, a hierarchical doubling-based index for graphs having large GEDs, defined in Def. 4.1. Then, we describe how NetDAG is constructed from the database and present its query algorithm. Finally, we compare NetDAG with other existing doubling-based indexes to demonstrate that it achieves the lowest query time complexity.

Definition 4.1 (NetDAG)**.** Given a graph database of N points $\mathcal{D} = \{p_0, p_1, \ldots, p_{N-1}\}$, a threshold $\alpha > 0$, a maximum similarity threshold τ with $0 \leq \tau \leq \alpha$, and an integer $\phi \geq 0$, a NetDAG is a ϕ-layered DAG $H = (Y_1, Y_2, \ldots, Y_\phi, E)$, where:

- Each layer Y_i is an $(\alpha \cdot 2^i)$-net (Def. 2.4) of $\mathcal{D}$;
- $E = \{(p_j^{i+1}, p_k^i) \mid d(p_j^{i+1}, p_k^i) \leq 3\alpha \cdot 2^i + 2\tau\}$, where $i = 1, \ldots, \phi - 1$ and p_k^i denotes the point p_k in layer Y_i.

Remarks on notations. The top layer is Y_ϕ, and the leaf layer is Y_1. For any node $p_j^{i+1} \in Y_{i+1}$, its children are $\text{children}(p_j^{i+1}) := \{\, p_k^i \in Y_i \mid (p_j^{i+1}, p_k^i) \in E \,\}$.

Since a node in Y_i may be connected to multiple parent nodes in Y_{i+1}, H is a DAG. As in doubling-based indexes [6, 25, 36], queries are answered by traversing H top-down. Let $r = \alpha \cdot 2^i$ be the radius of the child layer. The edge range $3r + 2\tau$ is *wider* than that of some existing methods (*e.g.,* Cover Tree [6] uses $2r$). While this increases the number of children per node, it allows *the query algorithm to maintain only one pivot at each layer* rather than a set of candidates as in existing methods (Corollary 2), yielding the lowest per-layer query complexity (Theorem 4.6).

To observe how the $3r + 2\tau$ rule enables maintaining one pivot in the top-down traversal of the query algorithm, we present the invariant that *all answers lie in the ball* $B(p^{i+1},\, 2r + 2\tau)$ *centered at the current pivot* (Theorem 4.2), where the parent radius contributes $2r$ and the 2τ margin keeps the answer ball $B(Q, \tau)$ covered under the triangle inequality. To maintain the invariant in the child layer (Y_i), an edge of the current pivot must reach a child p^i such that $B(p^i,\, r + 2\tau)$ covers the answer ball, *i.e.,* $d(p^i, Q) \leq r + \tau$. Since the current pivot satisfies $d(p^{i+1}, Q) \leq 2r + \tau$, the triangle inequality gives $d(p^{i+1}, p^i) \leq (2r + \tau) + (r + \tau) = 3r + 2\tau$. Hence, we only need to search for p^{i+1}'s children within $3r + 2\tau$, and such p^i always exists (by Corollary 2 in Appx. A.1 [20]), when there exists an answer to the query.

4.1 Index Construction

We construct NetDAG by following the greedy permutation of Net-Tree [25]:

(1) **Greedy permutation.** At each iteration (at layer i), we select the farthest unselected point from the current centers $\langle p_0, \ldots, p_{k-1} \rangle$ as the next center p_k. We use GREED [45] to compute approximate GEDs efficiently. The exact GED computation is not adopted, as it is known that the exact computation takes significantly longer when GEDs are large.

(2) **Layer and edge construction.** Note that its radius r_k, *i.e.,* the distance to the nearest earlier center, is non-increasing. When the radius first crosses $\alpha \cdot 2^i$ (*i.e.,* $r_{k-1} > \alpha \cdot 2^i \geq r_k$), the prefix $\langle p_0, \ldots, p_{k-1} \rangle$ forms an $(\alpha \cdot 2^i)$-net, *i.e.,* the layer Y_i. Then, we connect the new layer Y_i ($i \geq 1$) to its (already-generated) parent layer Y_{i+1} by our proposed rule (Lemma A.6) as follows: for each center $p_j^{i+1} \in Y_{i+1}$, every $p_\ell^i \in Y_i$ with $d(p_\ell^i, p_j^{i+1}) \leq 3\alpha \cdot 2^i + 2\tau$ becomes a child of p_j^{i+1}.

(3) **Termination.** When r_k is smaller than or equal to α, the construction stops; $\langle p_0, \ldots, p_{k-1} \rangle$ forms the α-net Y_0, which is the root set of EPF (Sec. 5). Similarly, each point in Y_0 is set as a child of the center in Y_1 within $3\alpha + 2\tau$ (see Appx. A.1.3 [20] for details).

Example 4.1. We use Fig. 3(b) to (partially) illustrate the construction of the structure of NetDAG. (1) The construction initializes the top layer Y_ϕ with an arbitrary point p_0. It then greedily selects the point farthest from the selected centers, Here, p_1 is the farthest from $\{p_0\}$, p_2 from $\{p_0, p_1\}$, and p_3 from $\{p_0, p_1, p_2\}$, respectively. The nearest earlier center of all three is p_0, and the three distances satisfy $r_1 > r_2 > r_3 > \alpha \cdot 2^{\phi-1}$. When p_4 is selected, r_4 is the first radius that is at most $\alpha \cdot 2^{\phi-1}$. (2) The prefix $\langle p_0, \ldots, p_3 \rangle$ forms $Y_{\phi-1}$, which is an $(\alpha \cdot 2^{\phi-1})$-net. Each of $p_0^{\phi-1}$, $p_1^{\phi-1}$, $p_2^{\phi-1}$, and $p_3^{\phi-1}$ within $3\alpha \cdot 2^{\phi-1} + 2\tau$ of p_0^ϕ becomes its child. The construction repeats for the lower layers and (3) terminates when $r_k \leq \alpha$, where the final prefix ($\langle p_0, \ldots, p_{k-1} \rangle$) forms the α-net Y_0.

Analysis. By following the analysis of Net-Tree's construction, the time complexity of the construction is $O(\lambda_\alpha^{O(1)} N\phi)$, where λ_α is the doubling constant down to radius α. The proofs are given in Appx. A.1.

4.2 Querying via Giant Steps

The giant-step query traverses the layers from Y_ϕ to Y_1, selecting a pivot at each layer. Let $\mathcal{A} = \{g \in \mathcal{D} \mid d(g, Q) \leq \tau\}$ be the answer set. At each layer Y_i, $\mathcal{A}$ is guaranteed to be contained in a ball centered at the pivot p with radius $\alpha \cdot 2^i + 2\tau$. The distance from

Algorithm 1: Giant-Step Query Processing on NetDAG

Input: H: NetDAG $(Y_1, \ldots, Y_\phi, E)$; Y_0: α-net; Q: query graph; τ: threshold
Output: $\mathcal{P}$: set of entry points in Y_0, or $\emptyset$ if no answer exists

1 $\mathcal{P} \leftarrow \emptyset$;
2 $p \leftarrow$ root of NetDAG; // initialize pivot
3 **for** $i = \phi - 1$ to 1 **do** // from root layer to leaf layer
4 $found \leftarrow$ **false**;
5 **for** p_j^i such that $(p, p_j^i) \in E$ **do**
6 **if** $d(p_j^i, Q) \le \alpha \cdot 2^i + \tau$ **then**
7 $p \leftarrow p_j^i$; // giant step: roughly halve $d(p, Q)$
8 $found \leftarrow$ **true**;
9 **break**;
10 **if** not $found$ **then return** $\emptyset$;
11 **for** p_j^0 such that $d(p, p_j^0) \le 3\alpha + 2\tau$ **do** // collect candidate balls
12 **if** $d(p_j^0, Q) \le \alpha + \tau$ **then**
13 $\mathcal{P} \leftarrow \mathcal{P} \cup \{p_j^0\}$;
14 **return** $\mathcal{P}$;

the pivot to Q decreases from at most $\alpha \cdot 2^\phi + \tau$ at the top to at most $2\alpha + \tau$ at the leaf, *roughly halving at each layer*. Alg. 1 presents the query algorithm for NetDAG. Starting with the root of NetDAG as the initial pivot p in the top layer Y_ϕ (Line 2), it iteratively traverses the layers. At each layer i, it searches for children of the current pivot p that satisfy the distance constraint $d(p_j^i, Q) \le \alpha \cdot 2^i + \tau$ (Lines 5–6), Alg. 1 takes the first child that satisfies the distance constraint as the new pivot and proceeds to the next layer (Lines 7–9). Each such step requires distance computations and roughly halves $d(p, Q)$, rapidly narrowing the search space (Fig. 4(b)). If no such child exists, the algorithm returns empty (Line 10). Upon reaching the leaf layer Y_1, the algorithm collects the points p_j^0 in Y_0 satisfying $d(p, p_j^0) \le 3\alpha + 2\tau$ and $d(p_j^0, Q) \le \alpha + \tau$ as candidate ball centers $\mathcal{P}$ for the subsequent EditPathForest search (Lines 11–13).

Theorem 4.2. *In Alg. 1, at each layer i ($i = \phi, \ldots, 1$), the pivot p satisfies $\mathcal{A} \subseteq B(p, \alpha \cdot 2^i + 2\tau)$.*

Proof. We prove by induction on i from ϕ to 1.

Base case ($i = \phi$). Since the top layer $Y_\phi = \{p\}$ is an $(\alpha \cdot 2^\phi)$-net of $\mathcal{D}$, every query answer is within distance $\alpha \cdot 2^\phi$ from p, so $\mathcal{A} \subseteq B(p, \alpha \cdot 2^\phi + 2\tau)$.

Inductive step. Assume $\mathcal{A} \subseteq B(p, \alpha \cdot 2^{i+1} + 2\tau)$ at layer $i + 1$ for some $i + 1 \ge 2$. If $\mathcal{A}$ is empty, the claim holds trivially. Otherwise, Corollary 2 ensures that there exists a child p' of p satisfying $d(p', Q) \le \alpha \cdot 2^i + \tau$. By the triangle inequality, for any $g \in \mathcal{A}$, $d(g, p') \le d(g, Q) + d(Q, p') \le \tau + (\alpha \cdot 2^i + \tau) = \alpha \cdot 2^i + 2\tau$, so $\mathcal{A} \subseteq B(p', \alpha \cdot 2^i + 2\tau)$. □

Theorem 4.3. *At layer i of Alg. 1, if a child p' of the current pivot p satisfies $d(p', Q) > \alpha \cdot 2^i + \tau$, then the child ball $B(p', \alpha \cdot 2^i)$ contains no query answer $g \in \mathcal{A}$.*

Proof. For any $g \in \mathcal{A}$, by the triangle inequality, $d(p', g) \ge d(p', Q) - d(g, Q) > \alpha \cdot 2^i$, so $g \notin B(p', \alpha \cdot 2^i)$. □

Remark. Theorem 4.3 guarantees the correctness of returning $\emptyset$ in Line 10 of Alg. 1. By Theorem 4.2, $\mathcal{A} \subseteq B(p, \alpha \cdot 2^{i+1} + 2\tau)$, which by Lemma A.6 is covered by its child balls. If no child p' satisfies $d(p', Q) \le \alpha \cdot 2^i + \tau$, then by Theorem 4.3, none of these child balls contains any $g \in \mathcal{A}$, so $\mathcal{A} = \emptyset$.

Table 2: Comparison of NetDAG and representative doubling-based methods (radius ratio 2; NDC:# of dist. computations).

Index	Child range	Max. # children	Traversal proceeds to	Max. cand. set size	O(NDC/layer)
NetDAG	$3r + 2\tau$	$\lambda_\alpha^{\log_2 20}$ (or c_α^6)	ONE child	1	$\lambda_\alpha^{\log_2 20}$ (or c_α^6)
Net-Tree [25]	$4r$	λ_α^4	child set	$\lambda_\alpha^{\log_2 52}$	$\lambda_\alpha^{4+\log_2 52}$
Navigating Nets [36]	$8r$	λ_α^5	child set	$\lambda_\alpha^{\log_2 28}$	$\lambda_\alpha^{5+\log_2 28}$
Cover Tree [6]	$2r$	c_α^4	child set	c_α^5	c_α^9

Theorem 4.4. *Let p be the pivot returned by Alg. 1 at layer 1, and let $\mathcal{P} = \{ p_j^0 \in Y_0 \mid d(p, p_j^0) \le 3\alpha + 2\tau$ and $d(p_j^0, Q) \le \alpha + \tau \}$. Then $\mathcal{A} \subseteq \bigcup_{p_j^0 \in \mathcal{P}} B(p_j^0, \alpha)$.*

Proof. For any $g \in \mathcal{A}$, by Theorem 4.2 with $p \in Y_1$, $d(g, p) \le 2\alpha + 2\tau$. Since Y_0 is an α-net, there exists $p_j^0 \in Y_0$ with $d(g, p_j^0) \le \alpha$, so $g \in B(p_j^0, \alpha)$. By the triangle inequality, $d(p, p_j^0) \le d(p, g) + d(g, p_j^0) \le (2\alpha + 2\tau) + \alpha = 3\alpha + 2\tau$, and $d(p_j^0, Q) \le d(p_j^0, g) + d(g, Q) \le \alpha + \tau$, so $p_j^0 \in \mathcal{P}$ and $g \in \bigcup_{p_j^0 \in \mathcal{P}} B(p_j^0, \alpha)$. □

Example 4.5. To illustrate the giant-step query (Fig. 3(b)), we set $\phi = 3$, giving three layers Y_ϕ, $Y_{\phi-1}$, and Y_1. Let the root pivot be $p = p_0^\phi \in Y_\phi$, which satisfies $d(p, Q) \le \alpha \cdot 2^\phi + \tau$. Then $B(p, \alpha \cdot 2^\phi + 2\tau)$ covers $B(Q, \tau)$ (contains $\mathcal{A}$). By Corollary 2, the algorithm selects a child $p_2^{\phi-1} \in Y_{\phi-1}$ as the new pivot p, satisfying $d(p, Q) \le \alpha \cdot 2^{\phi-1} + \tau$. The ball $B(p, \alpha \cdot 2^{\phi-1} + 2\tau)$ still covers $B(Q, \tau)$. Similarly, the algorithm selects $p_5^1 \in Y_1$ as the new pivot p, satisfying $d(p, Q) \le 2\alpha + \tau$. The ball $B(p, 2\alpha + 2\tau)$ covers $B(Q, \tau)$. The children p_j^0 of p in Y_0 satisfying $d(p_j^0, Q) \le \alpha + \tau$ are collected as candidate ball centers for the EditPathForest search (Theorem 4.4).

Theorem 4.6 (Time Complexity of Querying NetDAG). *Given a NetDAG with ϕ layers, Alg. 1 completes in $O(\lambda_\alpha^{\log_2 20} \cdot \phi)$ distance computations (NDCs).*

Proof idea. At each layer the pivot has at most $\lambda_\alpha^{\log_2 20}$ children (detailed in Corollary A.2 of Appx. A), and the algorithm checks at most this many of them to find the next pivot. Since there are ϕ layers, the overall complexity is $O(\lambda_\alpha^{\log_2 20} \cdot \phi)$. A detailed proof is deferred to Appx. A.

Discussion. We summarize our comparison of NetDAG with representative doubling-based indexes for non-graph data [6, 25, 36] in Tab. 2. We assume all methods build a hierarchical structure above the same α-net Y_0 with the parent-to-child radius ratio 2. The radius of layer i is $r_i = \alpha \cdot 2^i$, written as r in the table. For methods that give only asymptotic bounds of the form $\lambda^{O(1)}$, we derive specific constants in Appx. A.1.2 [20].

5 EDIT PATH FOREST: PATH-BASED INDEX FOR GRAPHS HAVING SMALL GEDS

Recall from Figs. 1(a) and 5 that when the GED between graphs is small, doubling-based indexes can be inefficient. To address this, we exploit the fact that graphs with small GEDs differ by only a few edit operations, and their edit paths from a common root share prefixes. Based on these, we propose EditPathForest (EPF), a prefix-tree-based index that shares common edit paths among nearby graphs, enabling path-based pruning, GED computation reuse, and search

in *small steps*. EPF is a set of EditPathTrees (EPTs). Each EPT has a root G_{root} that is a center in the α-net Y_0. EPT indexes the graphs that (i) are covered by the ball $B(G_{\text{root}}, \alpha)$ and (ii) have G_{root} as their nearest center in Y_0. EditPathTree (EPT) is defined as follows.

Definition 5.1. Given a set of graphs $\mathcal{C} = \{G_1, G_2, \ldots, G_m\}$ that are covered by $B(G_{\text{root}}, \alpha)$, a EditPathTree, denoted as $\mathcal{T}$, is a prefix tree rooted at G_{root} (which is a graph in $\mathcal{C}$) that indexes the graph edit paths (GEPs) from G_{root} to the graphs in $\mathcal{C}$, as follows.

- Each node G_i in $\mathcal{T}$ is a graph, and each edge represents the application of edit operation(s) from the parent graph to the child graph. A path from G_{root} to G_i is the sequence of edit operations that transforms G_{root} into G_i.
- Each node G_i in $\mathcal{T}$ that belongs to $\mathcal{C}$ is called a *data node*; otherwise, it is a *non-database node*.

Construction. The EPT is constructed in the three main steps. (1) **GEP computation.** For each data graph G_i, the GEP from the root G_{root} to G_i is extracted from the exact GED computation $d(G_{\text{root}}, G_i)$ by AStar-BMao [9]: the edit operations induced by the returned vertex mapping form the GEP, and a canonical ordering of operations is employed to maximize sharing in EPT. (2) **Prefix sharing.** GEPs that share a prefix of operations are merged in the prefix tree. (3) **Path compression.** Single-child non-database nodes are compressed into single edges. Hence, fewer GED computations are needed in query processing. The construction algorithm is presented in Alg. 6 in Appx. A.2 [20].

We note that the GEPs from G_{root} to all data graphs $G_i \in \mathcal{C}$ collectively require $\sum_{G_i \in \mathcal{C}} d(G_{\text{root}}, G_i)$ edit operations (edges in EPF). With (1) *prefix sharing* and (2) *path compression*, the edge number is reduced and the compression rates are 14.0%, 11.5%, 12.2%, and 18.4% on AIDS, PubChem, Chemical1M, and SYN, respectively.

Let $\mathcal{C}_p \subseteq \mathcal{D}$ be the data graphs indexed by the EPT rooted at $p \in Y_0$. By Corollary 3 in Appx. A.2 [20], the total space of EPF is $O(\sum_{p \in Y_0} |\mathcal{C}_p| \cdot |E(p)| + \sum_{G \in \mathcal{D}} |E(G)|)$. Alg. 6 computes one GEP per data graph (Line 2), so the construction performs N exact GED computations.

Example 5.1. Fig. 3 (c) is an example of EPT. G_{root} is the center of the ball. $\{G_1, G_2, G_3, G_4, G_5, G_6\}$ are the data graphs. G_2 is a data graph in $\mathcal{C}$ but is not a leaf node. G_9 is a *compressed node* that represents two edit operations. The EPT does not contain all data graphs in the ball, since some may be closer to another center and are assigned to other EPTs.

Remarks. There is a subtle technical detail at the boundary of NetDAG and EPF. We briefly describe how NetDAG is connected to EPF. As shown in Sec. 4.1, during the construction of NetDAG, the greedy permutation generates the leaf layer of NetDAG (*i.e.,* Y_1), as well as an α-net Y_0. As in Def. 4.1, $p^1 \in Y_1$ has an edge to $p^0 \in Y_0$ that satisfies $d(p^0, p^1) \leq 3\alpha + 2\tau$. For each p^0's ball, we build an EPT rooted at p^0. When the query algorithm (Sec. 4.2) reaches p^1, the EPTs of its children are searched further.

Index updates. Gisma can be updated as follows. To insert a graph G, we locate its nearest α-net center $p^0 \in Y_0$ by querying NetDAG, compute the edit path from p^0 to G, and merge it into the EPT rooted at p^0. To delete a graph G, we mask it in its EPT in $O(1)$ time. Gisma is rebuilt when its efficiency notably degrades.

Algorithm 2: Small-step Query Processing on EPT

```
Input: T: EditPathTree; Q: query graph; τ: threshold
Output: A: set of data graphs within distance τ
1  A ← ∅;
2  initialize a stack S with (T.root, 0)
3  while S ≠ ∅ do
4      (G, d_lb) ← pop(S);    // small step: traverse along edit paths
5      d̄_G ← max{ d(G, G_d) | G_d is a descendant of G};  // precomputed
6      if d_lb ≤ τ then
7          d_lb ← d(G, Q);                          // 5.1.3 compute GED
8      if d_lb − d̄_G > τ then
9          continue;                              // 5.1.1 subtree pruning
10     if G is a data graph and d_lb ≤ τ then
11         add G to A;
12     for G_c is a child of G do
13         d_lb(G_c) ← d_lb − d(G, G_c) ;    // 5.1.2 lower-bound propagation
14         push((G_c, d_lb(G_c)), S);
15 return A;
```

5.1 Querying via Small Steps

The small-step query is restricted to the candidate balls identified by Alg. 1 on the NetDAG, and each of them is queried independently. Next, we describe how one EPT is queried by a depth first traversal (Alg. 2). To improve query efficiency, we introduce three optimizations, namely, (1) EPT *subtree pruning*; (2) *lower-bound pruning and its propagation*; and (3) *search tree reuse in* GED *computations*.

5.1.1 EPT *Subtree Pruning.* To reduce the number of GED computations, we propose EPT *subtree pruning* to prune the EPT subtrees that cannot contain answers. Let $\bar{d}_G$ denote the *upper bound* of GED from a graph G in its EPT to any of its descendants. As $\bar{d}_G$ of each graph depends only on its EPT structure, $\bar{d}_G$ is precomputed and stored during construction and retrieved during query processing (Line 5). The minimum possible distance between any graph in the subtree rooted at G and the query Q is $d(G, Q) - \bar{d}_G$. By the triangle inequality of GED, $d(G', Q) \geq d(G, Q) - d(G, G')$, where G' is an arbitrary descendant of G. If $d(G, Q) - \bar{d}_G > \tau$, then $d(G', Q) > \tau + \bar{d}_G - d(G, G') \geq \tau$. Hence, Lines 8–9 ensure that Alg. 2 does not traverse the subtree rooted at G.

5.1.2 Lower-bound Pruning and its Propagation. For an edge (G_p, G_c) of EPT, once $d(G_p, Q)$ is computed, we define $\underline{d}(G_c, Q) = d(G_p, Q) - d(G_p, G_c) \leq d(G_c, Q)$ as a lower bound of $d(G_c, Q)$. If $\underline{d}(G_c, Q) > \tau$, then G_c cannot be an answer and its GED computation is skipped (Line 6). When $d(G_p, Q)$ is not computed because G_p was itself pruned by its lower bound, the lower bound can still be *propagated*: we update $\underline{d}(G_c, Q) = \underline{d}(G_p, Q) - d(G_p, G_c)$ (Line 13). Similarly, if $\underline{d}(G_c, Q) > \tau$, G_c cannot be an answer. A GED computation is performed only when the propagated lower bound does not exceed the query threshold (Lines 6–7), thereby minimizing the total number of GED computations.

Putting these together, Alg. 2 presents the query procedure on the EPT. It performs a depth first traversal while applying subtree pruning of Sec. 5.1.1 (Lines 8–9), and lower-bound pruning (Line 6) and lower-bound propagation of Sec. 5.1.2 (Line 13).

Example 5.2. Consider the EPT shown in Fig. 3(c). The query threshold is set to $\tau = 2$. G_{root} has $d(G_{\text{root}}, Q) = 4$ and $\bar{d}_{G_{\text{root}}} = 3$, giving $d(G_{\text{root}}, Q) - \bar{d}_{G_{\text{root}}} = 1 \leq \tau$, so its subtree remains to be explored. For the child G_7, the lower bound propagated from G_{root} is $\underline{d}(G_7, Q) = 3$. Since $\underline{d}(G_7, Q) - \bar{d}_{G_7} = 1 < \tau$, the subtree of G_7 is not

pruned and continues to be searched. Meanwhile, as $\underline{d}(G_7, Q) > \tau$, the distance computation at G_7 is skipped by lower-bound pruning (Line 6). For its child G_2, the lower bound is propagated as $\underline{d}(G_2, Q) = \underline{d}(G_7, Q) - d(G_7, G_2) = 2 \leq \tau$, which leads to computing $d(G_2, Q) = 6$. Since $d(G_2, Q) - \bar{d}_{G_2} = 5 > \tau$, the subtree rooted at G_2 is pruned (Line 8).

5.1.3 Search Tree Reuse in GED *Computations.* We first recall that the A* search algorithm [8, 9, 33, 51] has been widely adopted for computing GED, which formulates GED computation as a vertex-matching problem. In these methods, A* organizes (partial) vertex mappings into a *search tree*, and expands this tree to obtain a full mapping with the minimum edit cost. Each *state* of the search tree corresponds to *a (partial) mapping between the two graphs.*

It should be noted that for two similar graphs G and G' (*i.e.,* having small $d(G, G')$) with the same number of vertices, the search trees for $d(G, Q)$ and $d(G', Q)$ often have large overlaps. To ensure efficiency, we consider the case where *the parent and child graphs in* EPT *have the same number of vertices*, and propose to reuse the search tree constructed during the computation of $d(G, Q)$ to accelerate constructing that of $d(G', Q)$.

To achieve this, we retain from the search tree of $d(G, Q)$ *only the states that are potentially useful for computing* $d(G', Q)$. We denote by $d(G, Q|s)$ the edit distance from G to Q that completes the partial mapping at state s, and by $\underline{c}(G, Q|s)$ its lower bound. It should be remarked that $\underline{c}(G, Q|s) \leq \underline{c}(G, Q|s')$ if s' is a leaf in the subtree rooted at s in the search tree. After finishing $d(G, Q)$, we construct a *search tree* S_G for reuse by keeping every leaf (a.k.a *frontier*) state s of the search tree such that $\underline{c}(G, Q|s) \leq \tau + d(G, G')$ (*i.e.,* $\underline{c}(G, Q|s) - d(G, G') \leq \tau$), together with all its ancestor states. Due to the triangle inequality, given s, we have $d(G', Q|s) \geq d(G, Q|s) - d(G, G') \geq \underline{c}(G, Q|s) - d(G, G')$. Hence, if $\underline{c}(G, Q|s) - d(G, G') > \tau$, then $d(G', Q|s) > \tau$ and the search subtree rooted at s can be pruned. In addition, resuming the search from the frontiers of S_G does not miss any answers.

Alg. 3 presents $d(G', Q)$ computation with the reuse of search tree S_G. For subsequent presentation, when the graphs are clear from the context, we simply write $d(s)$ and $\underline{c}(s)$, at state s. Line 1 initializes the minimum edit cost $c_{\min}$ as τ. In Line 2, H is a priority queue that maintains the states that may lead to a full mapping at a cost at most $c_{\min}$. Alg. 3 retrieves the frontier states of S_G (Line 3) and computes their lower bounds $\underline{c}$, *e.g.,* by using [9, 51] (Line 5). In Lines 6–7, if $\underline{c}(s) \leq c_{\min}$, the frontier state s is pushed into H. In practice, this reduces the value of $c_{\min}$ faster, and consequently, reduces the number of states pushed into H. In Lines 8–16, Alg. 3 resumes the search from the states in H. It pops a state s from H (Line 9) that may lead to a smaller cost than $c_{\min}$. If s corresponds to a full mapping, $c_{\min}$ is updated (Lines 10–11); otherwise, Alg. 3 expands s and pushes children s's, where $\underline{c}(s') \leq c_{\min}$ (Lines 13–16). The search terminates once H is empty (Line 8), and it returns the $c_{\min}$ found (Line 17).

Example 5.3. Fig. 6 shows an example of search tree reuse. We have computed $d(G_9, Q)$ and obtained the search tree S_{G_9}, as illustrated in Fig. 6(a), which is the input to Alg. 3. G' is G_6, which is a child of G_9 with $d(G_6, G_9) = 1$, and $\tau = 2$. Line 3 computes the frontiers of S_{G_9} (the thick boxes). We note that s_9, having $\underline{c} = 4$, is not included in S_{G_9} because $\underline{c} > \tau + d(G_6, G_9) = 3$.

Algorithm 3: GED Computation with Search Tree Reuse

Input: S_G: a reused search tree from (Q, G); Q: query graph; G': data graph, where G and G''s size are the same; τ: threshold
Output: $d(G', Q)$: the GED between G' and Q

1 $c_{\min} \leftarrow \tau$ $\quad \Delta_{c_{\min}} \leftarrow$ false ; `// initialize min. edit cost`
2 $H \leftarrow$ an empty priority queue, in ascending order of $\underline{c}(s)$;
3 $L \leftarrow$ frontiers of S_G;
4 **foreach** $s \in L$ **do**
5 $\quad \underline{c}(s) \leftarrow$ `App-BMao-resume`(s, G', Q); `// update lower bound cost at` s, `e.g., [9, 51]`
6 $\quad$ **if** $\underline{c}(s) \leq c_{\min}$ **then**
7 $\quad\quad$ push(s, H);
8 **while** $H \neq \emptyset$ and $\underline{c}(\text{top}(H)) \leq c_{\min}$ **do**
9 $\quad s \leftarrow \text{pop}(H)$;
10 $\quad$ **if** s corresponds to a full mapping (from G' to Q) **then**
11 $\quad\quad c_{\min} \leftarrow \underline{c}(s)$ $\quad \Delta_{c_{\min}} \leftarrow$ true; `// update min. edit cost`
12 $\quad\quad$ **continue**;
13 $\quad$ **foreach** $s' \in \text{child}(s)$ **do**
14 $\quad\quad \underline{c}(s') \leftarrow$ `App-BMao-resume`(s', G', Q); `// update lower bound cost at` s'
15 $\quad\quad$ **if** $\underline{c}(s') \leq c_{\min}$ **then**
16 $\quad\quad\quad$ push(s', H);
17 **if** $\Delta_{c_{\min}}$ **then return** $c_{\min}$ **else return** `"undefined"`

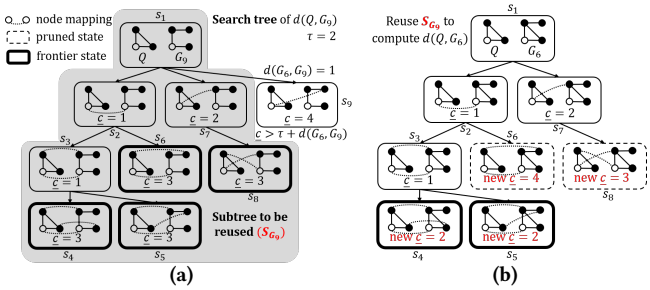


Figure 6: An illustration of search tree reuse using the example in Fig. 3(c). (a) shows the search tree constructed for $d(G_9, Q)$ and the search subtree S_{G_9} for reuse in computing $d(G_6, Q)$, where G_6 is a child of G_9 with $d(G_6, G_9) = 1$ and τ is set to 2, and (b) shows how S_{G_9} is reused to compute $d(G_6, Q)$. The state with $\underline{c} = 4$ is pruned because $\underline{c} > \tau + d(G_6, G_9)$.

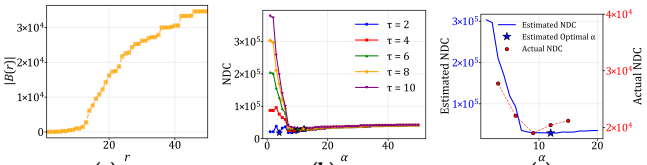
Fig. 6(b) shows how S_{G_9} is used to compute $d(G_6, Q)$. In Lines 4–5, the search resumes computing $\underline{c}$ at each frontier state. The updated $\underline{c}$s are shown in red in Fig. 6(b). In Line 6, s_6 is pruned because $\underline{c}(s_6)$=4 > $c_{\min}$=2. s_4, s_5, and s_8 are further checked (Lines 6–7). The $\underline{c}$s of s_4 and s_5 are smaller than $c_{\min}$ (Line 8), and s_4 and s_5 correspond to a full mapping from G_6 to Q (Lines 10–11), the minimum edit cost is updated to 2. Also, this indicates that G_6 is an answer within τ. The search continues to check other states (Line 13). In Line 8, since $\underline{c}(s_8)$=3 > $c_{\min}$=2, s_8 is not examined further.

Due to space restrictions, we present the complexity analyses of the three optimizations in Appx. 7 [20].

6 COST MODEL FOR Gisma CONSTRUCTION

The query efficiency bottleneck of graph similarity search in Gisma is GED computation. The radius threshold α between `NetDAG` and `EditPathForest` introduces a tradeoff in NDC between the two stages. In this section, we propose a cost model to find the optimal α that minimizes the total NDC during query processing.

The NDC of querying `NetDAG` can be estimated as follows. At each layer i, the children's $(\alpha \cdot 2^i)$-balls jointly cover $B(\text{pivot}, \alpha \cdot$



(a) (b) (c)

Figure 7: Illustrations of $|B(r)|$ and the estimated NDC(α) under different α values on the AIDS dataset, including the estimated curve under $\tau = 8$ and measured NDC at selected α.

$2^{i+1}+2\tau$) (Lemma A.6), which requires $\texttt{Cover}(\alpha\cdot 2^i,\ \alpha\cdot 2^{i+1}+2\tau)$ children, where $\texttt{Cover}(r, R)$ denotes the minimum number of r-balls needed to cover a ball of radius R. In the worst case, the algorithm computes the distance from Q to every child. The total NDC incurred in NetDAG (including the NDC to traverse from NetDAG to EPF) is $\sum_{i=0}^{\phi-1} \texttt{Cover}(\alpha \cdot 2^i,\ \alpha \cdot 2^{i+1} + 2\tau)$.

After querying NetDAG, the candidate balls are $B(p_j^0, \alpha)$ with centers $p_j^0 \in Y_0$ satisfying $d(p_j^0, Q) \le \alpha + \tau$ (Theorem 4.4). By the triangle inequality, all graphs g in these balls satisfy $d(g, Q) \le d(g, p_j^0) + d(p_j^0, Q) \le \alpha + (\alpha + \tau) = 2\alpha + \tau$. In the worst case, the NDC incurred in EPF is bounded by $|B(Q, 2\alpha + \tau)|$, the number of data graphs within distance $2\alpha + \tau$ of Q.

We combine the NDC of NetDAG and EPF into a cost model (Eq. (1)) to find the optimal α.

$$\text{NDC}(\alpha) = \sum_{i=0}^{\phi-1} \texttt{Cover}(\alpha \cdot 2^i,\ \alpha \cdot 2^{i+1} + 2\tau) \ +\ |B(2\alpha + \tau)| \qquad (1)$$

Since $\texttt{Cover}(\alpha \cdot 2^i, \alpha \cdot 2^{i+1} + 2\tau)$ cannot be directly computed, we approximate it as follows. Since τ is small relative to $\alpha \cdot 2^{i+1}$, we scale $\texttt{Cover}(\alpha \cdot 2^i, \alpha \cdot 2^{i+1})$ proportionally to the ball sizes. Note that $\texttt{Cover}(\alpha \cdot 2^i, \alpha \cdot 2^{i+1}) = \max_p \lambda(p,\ \alpha \cdot 2^i)$. Then, we have:

$$\text{NDC}(\alpha) \approx \sum_{i=0}^{\phi-1} \frac{|B(\alpha\cdot 2^{i+1}+2\tau)|}{|B(\alpha\cdot 2^{i+1})|} \max_p \lambda(p,\ \alpha \cdot 2^i) + |B(2\alpha + \tau)|. \qquad (2)$$

When α is small, the first term contains large summands such as $\max_p \lambda(p, \alpha)$ and $\max_p \lambda(p, 2\alpha)$, which are large due to the high doubling constant at small radii (as observed in Sec. 3). When α is large, $|B(2\alpha + \tau)|$ approaches the entire database size, making the second term dominant. An optimal α minimizes NDC(α).

Example 6.1. Based on Eq. (2), Figs. 5 and 7(a), we plotted Fig. 7(b) to illustrate how the NDC varies with α under different τ values on the AIDS dataset. The results show that when $\tau = 2$, the minimum NDC occurs at $\alpha = 4$, while for larger thresholds ($\tau = 4$–10), the optimal α is between 9 and 12. In particular, both $\tau = 8$ and $\tau = 10$ reach their minima of α at around 12.

7 EXPERIMENTAL EVALUATION

In this section, we present experiments to investigate the performance of Gisma. The code is available on Github.[3] We present the experimental settings in Sec. 7.1, report the overall performance in Sec. 7.2, and provide detailed evaluation in Sec. 7.3.

[3]https://github.com/McKayGONG/Gisma

Table 3: Statistics of datasets for experiments

Dataset	$\lvert\mathcal{D}\rvert$	Avg. $\lvert V\rvert$	Avg. $\lvert E\rvert$	# v_label	Used in
AIDS	42.7K	25.6	27.5	62	[9, 10, 32, 42, 44, 51]
PubChem	22.8K	48.1	50.6	10	[9, 32, 42]
Chemical1M	1M	24.0	25.8	94	-
SYN	1M	14.3	20.5	1	[9, 42]

7.1 Experimental Settings

Platform. We implemented Gisma in C++. We conducted experiments on a machine equipped with a dual 64-core AMD EPYC 7742 processor (2.25 GHz) and 2 TB RAM running Ubuntu OS.

Datasets. Following previous works [8, 9, 32, 42], we evaluated Gisma using both real-world and synthetic datasets. Specifically, we used two widely adopted real-world datasets, namely AIDS[4] and PubChem[5] [34], as well as a large-scale dataset, Chemical1M, which is also derived from the PubChem database. AIDS is a set of antiviral compounds, PubChem and Chemical1M are datasets of chemical compound graphs. To facilitate our scalability tests, we also generated a set of synthetic graphs (denoted as SYN) by using GraphGen[6]. Tab. 3 summarizes the statistics of these datasets.

Queries. We randomly selected 100 graphs from each dataset as query graphs and excluded them from the datasets. We followed [8, 9, 32] to report the average runtimes of all the queries.

Benchmark methods. We compared Gisma with the representative related methods. Although AStar-BMao [9] is designed for GED verification, its released code includes a filtering method. To ensure fair comparison with methods without filtering (*i.e.,* AppBMao and GEDHOT), we apply the same filtering method and denote them with "+".

- AStar-BMao+ [9]. AStar-BMao is the state-of-the-art (SOTA) for **exact** GED verification. It can answer graph similarity search queries by pairwise comparison of all data graphs and query graphs.
- App-BMao+ [51]. App-BMao is a recent method for computing *approximate* GED between two graphs. It adopts a similar A* search framework to AStar-BMao, but its implementation introduces a limit that terminates the search early, at the cost of accuracy.
- GEDHOT+ [10]. GEDHOT is a recent ensemble method for *approximate* GED estimation, reported as the most accurate among learning-based GED methods. It takes advantage of a supervised model GEDIOT and an unsupervised model GEDGW.
- GHashing [44]. GHashing is a recent learning-based method for *approximate* graph similarity search. It employs graph neural networks to learn semantic hash codes for candidate generation.
- LAN [42]. LAN is a recent approximate k-nearest neighbor (kNN) search method for graph databases based on proximity graphs. It proposes a learning-based initial node selection and neighbor pruning in query processing. We extend LAN to support similarity search by progressively increasing the search range.
- Nass [32]. Nass is the SOTA index-based solution for *exact* graph similarity search. During index construction, Nass precomputes pairwise GED among the graphs in the dataset within a certain GED range. During query processing, it uses the precomputed distances to iteratively reduce the candidate set by triangle inequality.
- Gisma. Gisma is our proposed index. During the construction of NetDAG, we use GREED [45] to compute approximate GED, as

[4]https://cactus.nci.nih.gov/download/nci/AID2DA99.sdz
[5]https://pubchem.ncbi.nlm.nih.gov
[6]https://www.cse.cuhk.edu.hk/~jcheng/graphgen1.0.zip

it is known that the exact computation takes significantly longer for the many large-distance pairs involved. GREED is currently the fastest approximate GED method and achieves a small mean absolute error (MAE).

For presentation simplicity, we refer to AStar-BMao+ and Nass as the exact methods and App-BMao+, GEDHOT+, GHashing, and LAN as *approximate methods*.

Parameters. Unless otherwise specified, the parameters are set as follows. For baseline methods, AStar-BMao+, Nass, GEDHOT+, GHashing, and LAN use their original default configurations, while App-BMao+ and Gisma are configured to achieve approximately 90% recall at each τ. The default value of α is 12 for AIDS, PubChem, and Chemical1M, and 6 for SYN, respectively.

7.2 Overall Performance

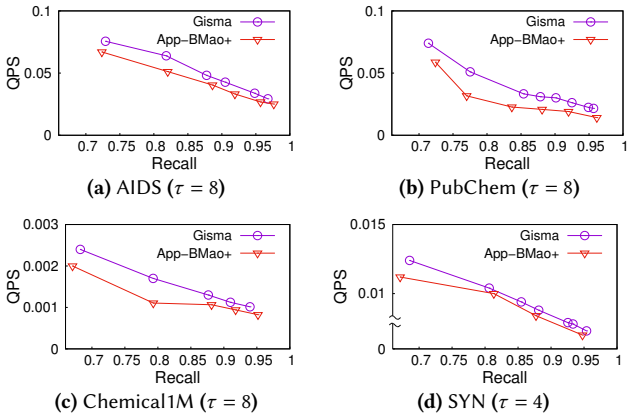
EXP-1. Overall evaluation on efficiency. We compare the efficiency of Gisma with the benchmark methods. Tab. 4 presents the overall runtime, along with recall and precision, across four benchmark datasets with various threshold values τ.

Across all datasets, at small τ Gisma remains competitive in efficiency , with precision at 100% and recall at ~95% or higher; it becomes the most efficient method as τ increases. In addition, on AIDS and PubChem datasets, Gisma reduces the runtime by 26.6%–35.9% compared to the second-best App-BMao+ at $\tau = 12$, and shows even greater improvements (62.7%–67.9%) over AStar-BMao+. This is particularly evident in Chemical1M, where the efficiency improves from 30.8% at $\tau = 4$ to 56.3% at $\tau = 10$ when compared to the exact methods. On the SYN dataset, Gisma maintains a consistent advantage over all methods at high thresholds.

Gisma continues to be efficient across all datasets when the τ values are large.[7] In contrast, Nass fails to build indices on the AIDS, PubChem, and SYN datasets when threshold τ is large, and it cannot build indices on Chemical1M. LAN and GHashing often cannot finish within 24 hours for large τ values. In particular, LAN performs full GED computation (without threshold-based pruning) at every routing step, which is costly when the visited graphs are far from the query. GEDHOT+ returns low-quality results across all settings, indicating that even the most accurate learning-based GED estimation cannot yet serve as a GED verifier.

EXP-2. Overall evaluation on effectiveness. Tab. 4 reports recall and precision of the methods, where recall is macro-averaged over queries that have at least one ground-truth answer, and precision is macro-averaged over queries that return at least one result. Gisma's precision is 100% and its recall is above 90% across all τs, matching the strongest baseline App-BMao+, whereas the learning- and hashing-based methods are less accurate: GEDHOT+ reaches higher recall only at low precision, and GHashing and LAN keep high precision, but either drop recall sharply on some datasets (*e.g.,* GHashing falls to 41.9% on AIDS) or match high recall only at a much higher query cost.

Fig. 8 compares the QPS-recall trade-off between Gisma and its best competitor App-BMao+. Across all four datasets, Gisma consistently achieves higher QPS than App-BMao+ at every recall level, especially on AIDS and PubChem where Gisma achieves 1.2–1.6× higher QPS. In other words, at any fixed recall, Gisma answers more queries per second than the strongest baseline. The QPS-recall comparison with all other methods is provided in Fig. 10 (appendix). Tab. 5 further highlights the speedup of Gisma over App-BMao+ at recalls of 0.80, 0.90, and ~0.95. Gisma is more efficient on all four datasets, by up to 1.64×.



(a) AIDS ($\tau = 8$) **(b)** PubChem ($\tau = 8$)
(c) Chemical1M ($\tau = 8$) **(d)** SYN ($\tau = 4$)

Figure 8: Comparison of QPS-recall curves

7.3 Detailed Evaluation of Gisma

In this section, we conduct an in-depth investigation of Gisma.

EXP-3. Index size and construction time. Tab. 6 reports the index sizes of different methods, along with the corresponding database sizes for reference. The index sizes of Gisma and LAN are of the same order of magnitude as the raw database sizes. Nass has smaller index sizes but fails to build indices for Chemical1M and SYN (CNB) due to $O(N^2)$ precomputing cost. Tab. 6 also reports the index construction time (in parentheses). Gisma builds its single index in 52 min on AIDS, 71 min on PubChem, 3.6 h on SYN, and 17.0 h on the million-graph Chemical1M, whereas LAN takes up to 92 h on AIDS, and Nass cannot build its index on the large datasets SYN and Chemical1M (CNB) and its cost grows sharply with the threshold (Appx. A.6 [20]).

EXP-4. Ablation study of Gisma. We conduct ablation experiments to analyze the contributions of the giant-step and small-step stages. We compare Gisma with the following three methods:

- Gisma-no-SS: It removes the small-step stage. It uses `NetDAG` to perform the giant-step stage to narrow down the search range, and then applies App-BMao+ to verify the graphs within this range.
- Gisma-no-GS: It removes the giant-step stage. It performs small-step traversal on all `EditPathTrees` without the pruning of the giant-step search.
- App-BMao+: It has neither the giant-step nor the small-step stage, *i.e.,* the original App-BMao+ method.

Tab. 8 presents the runtime comparison across various τs. The results show that both giant-step (GS) and small-step (SS) stages contribute to Gisma's efficiency. Compared with the App-BMao+ method (Gisma w/o GS and SS), Gisma achieves the smallest runtimes in most cases, with the runtime reduced by up to 35.9%, while App-BMao+ is slightly faster than others for very small τs.

[7] Applications such as production ligand discovery [41], and the search of molecules of the same biological activity (from Hert et al. benchmark [26] and ChEMBL [39]) and drugs [47] require searches with large τs. Further discussion can be found in Appx. A.8 [20].

Table 4: Runtime, recall, and precision of benchmarked methods under default settings with varying τ. Format: seconds per query (Recall%/Precision%); the exact methods Nass and AStar-BMao+ are simply marked "(exact)".

Dataset	Method	Seconds per query (Recall%/Precision%)					
		τ=2	4	6	8	10	12
AIDS	Nass	**0.0040** (exact)	0.189 (exact)	6.771 (exact)	172.4 (exact)	CNB	CNB
	AStar-BMao+	0.0048 (exact)	0.1132 (exact)	2.190 (exact)	37.81 (exact)	463.6 (exact)	6140 (exact)
	Gisma	0.0070 (99.65/100)	0.1041 (99.98/100)	**1.405** (98.46/100)	**20.41** (90.51/100)	**259.9** (90.07/100)	**2291** (90.09/100)
	App-BMao+	0.0046 (100/100)	**0.1019** (100/100)	1.752 (98.48/100)	25.04 (91.97/100)	278.2 (91.42/100)	3120 (91.53/99.92)
	LAN	69.77 (67.4/100)	105.9 (71.6/100)	206.8 (81.2/100)	923.2 (82.4/100)	2906 (69.3/100)	CNF
	GHashing	0.3861 (78.8/100)	0.5626 (70.6/100)	4.257 (64.0/100)	48.66 (56.3/100)	401.6 (48.6/100)	1885 (41.9/98.9)
	GEDHOT+	0.6304 (21.7/92.0)	6.766 (41.7/43.3)	45.36 (40.4/19.1)	108.4 (46.0/19.4)	178.6 (53.2/27.8)	261.4 (58.4/38.9)
		τ=2	4	6	8	10	12
PubChem	Nass	**0.0040** (exact)	**0.144** (exact)	3.946 (exact)	114.5 (exact)	CNB	CNB
	AStar-BMao+	0.0250 (exact)	0.2790 (exact)	4.816 (exact)	67.70 (exact)	711.5 (exact)	9634 (exact)
	Gisma	0.0214 (94.84/100)	0.2025 (96.55/100)	**2.445** (92.98/100)	**30.48** (90.77/100)	**323.6** (90.81/100)	**3093** (90.82/100)
	App-BMao+	0.0238 (100/100)	0.2511 (100/100)	3.853 (96.96/100)	44.82 (93.95/100)	426.9 (93.23/100)	4822 (93.16/99.14)
	LAN	326.1 (36.5/100)	486.0 (54.0/100)	664.3 (55.6/100)	962.7 (59.3/100)	1486 (49.1/98.8)	2345 (32.9/99.2)
	GHashing	0.2502 (96.4/100)	0.5605 (94.2/100)	4.978 (92.5/100)	68.57 (80.5/100)	821.0 (81.9/99.6)	5585 (75.3/98.6)
	GEDHOT+	0.4073 (14.6/24.1)	2.450 (68.3/22.5)	20.63 (85.1/7.6)	55.22 (86.1/4.3)	106.5 (84.0/3.0)	164.4 (77.4/2.6)
		τ=2	4	6	8	10	–
Chemical1M	Nass	CNB	CNB	CNB	CNB	CNB	–
	AStar-BMao+	0.1805 (exact)	5.805 (exact)	109.6 (exact)	1667 (exact)	18855 (exact)	–
	Gisma	0.1920 (99.84/100)	**4.018** (99.53/100)	**66.87** (98.67/100)	**801.3** (91.57/100)	**8247** (90.14/100)	–
	App-BMao+	**0.1715** (100/100)	5.225 (100/100)	87.76 (99.05/100)	956.7 (91.81/100)	11314 (92.24/100)	–
	LAN	76.79 (30.4/100)	655.4 (41.8/100)	CNF	CNF	CNF	–
	GHashing	7.797 (61.8/100)	13.04 (54.9/100)	126.5 (49.9/100)	5837 (42.1/100)	CNF	–
	GEDHOT+	30.61 (28.3/58.1)	289.9 (42.7/7.0)	993.4 (49.7/5.7)	2023 (61.0/10.4)	3132 (69.3/18.9)	–
		τ=1	2	3	4	5	6
SYN	Nass	**0.7340** (exact)	14.50 (exact)	CNB	CNB	CNB	CNB
	AStar-BMao+	0.8633 (exact)	8.117 (exact)	51.12 (exact)	165.1 (exact)	920.5 (exact)	2933 (exact)
	Gisma	0.8149 (98.96/100)	**7.237** (98.33/100)	**35.15** (98.15/100)	**129.2** (90.51/100)	**455.2** (90.78/100)	**1259** (90.29/100)
	App-BMao+	0.8201 (100/100)	7.305 (100/100)	40.90 (98.74/100)	142.8 (91.31/100)	552.3 (90.29/100)	1416 (90.18/100)
	LAN	31.86 (9.0/100)	65.23 (2.7/100)	850.6 (16.5/100)	CNF	CNF	CNF
	GHashing	433.7 (100/100)	1654 (98.5/100)	7152 (97.2/100)	35141 (99.9/100)	CNF	CNF
	GEDHOT+	69.23 (43.1/100)	338.3 (38.7/99.9)	1763 (33.2/96.2)	3431 (30.2/82.4)	5337 (27.6/61.4)	7192 (27.7/50.2)

Bold: best runtime; underline: second-best, among methods whose recall and precision are both $\geq$ 50%. CNB: cannot build index; CNF: cannot finish in 24h; "–": not measured. For Gisma, the large-τ entries ($\tau \geq 10$, or $\tau \geq 5$ for SYN) use a larger approximate-GED iteration budget; and all other entries are at default settings.

Table 5: Speedup of Gisma over App-BMao+ at various recalls: Speedup = QPS(Gisma)/QPS(App-BMao+) at the same recall.

Dataset \ recall	0.80	0.90	~0.95
AIDS	1.22×	1.17×	1.19×
PubChem	1.64×	1.51×	1.43×
Chemical1M	1.51×	1.17×	1.17×
SYN	1.04×	1.06×	1.06×

Table 6: Index sizes (MB) of different methods on the datasets, with the index construction time (in parentheses), and raw dataset sizes for reference. CNB: cannot build the index within one week.

Dataset	Gisma	LAN	Nass	Raw dataset
AIDS	91.6 (52 min)	49.1 (92 h)	28.1 (39.0 h)	20.6 MB
PubChem	53.5 (71 min)	45.9 (69 h)	0.8 (21.6 h)	20.8 MB
Chemical1M	1797.6 (17.0 h)	1099.6 (6.9 h)	CNB	454.7 MB
SYN	1451.8 (3.6 h)	768.9 (8.0 h)	CNB	309.6 MB

When comparing Gisma with Gisma-no-GS, Gisma achieves lower runtimes in most cases, with runtime reductions of up to 24.0% across different datasets and τs. Similarly, when comparing Gisma with Gisma-no-SS, which removes the small-step stage, the runtime reductions reach up to 33.7%. The giant-step stage is more effective for smaller τ values as it helps narrow down the search space early, while the small-step stage is increasingly effective with larger τ values. Specifically, as τ increases, the runtime difference between Gisma and Gisma-no-SS widens significantly (*e.g.,* up to 25.1% at τ=12 on AIDS).

Table 7: NDC of the giant steps, per query at the default τ (τ=4 for SYN, τ=8 otherwise).

Dataset	GS NDC	EPT NDC	Total NDC	Full scan (N)
AIDS	1,738	18,661	20,399	42,582
PubChem	2,359	8,334	10,693	22,694
Chemical1M	6,273	428,539	434,812	999,900
SYN	17,063	653,615	670,678	999,900

Tab. 7 quantifies how the giant step narrows the search. At the default τ, it uses only 0.6%–11% of a full scan in NDCs and prunes the search to 19%–62% of the EPTs. For example, on AIDS the giant step performs only 1,738 GED computations and enters 745 of the 3,399 EPTs (22%), instead of scanning all 42,582 graphs.

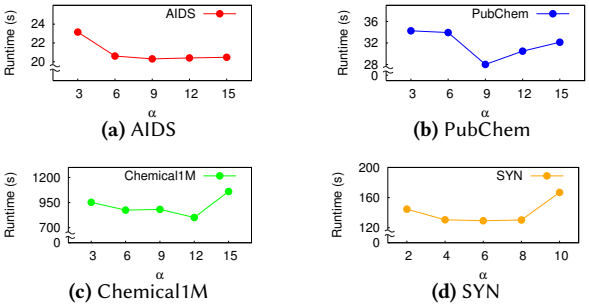
EXP-5. Ablation study of `EditPathForest` optimizations. Tab. 8 also evaluates the three optimizations for EPT query across τ, where each variant removes one optimization from Gisma: Gisma-no-Reuse (no search tree reuse), Gisma-no-SP (no subtree pruning), and Gisma-no-LP (no lower-bound propagation). Search tree reuse has the largest impact: Gisma-no-Reuse is the slowest variant at nearly all τ. Subtree pruning and lower-bound propagation give smaller but consistent speedups. At the smallest τ the per-query time is sub-second and the differences fall within run-to-run noise.

EXP-6. Cost model. We compare the trends of the estimated NDC(α) and the measured NDC on AIDS (τ = 8, Fig. 7(c)). Both curves indicate the same minimum region (α = 9–12), with a Pearson correlation of r = 0.97 (p-value = 0.0065); the model's selected α = 12 is within 7.4% of the best measured α (α = 9) in NDC. Hence, we can use the cost model to estimate the optimal-α region.

Table 8: Ablation study of Gisma (runtime in seconds). Due to space restrictions, we report AIDS and SYN here; the results on PubChem and Chemical1M are in Tab. 10 of Appx. A.3 [20].

Dataset	Method	Runtime (s)					
		τ=2	4	6	8	10	12
AIDS	Gisma	0.0070	0.1041	**1.405**	**20.41**	**259.9**	**2291**
	Gisma-no-GS	0.0062	0.1062	1.445	21.30	264.2	2357
	Gisma-no-SS	0.0059	0.1048	1.599	23.38	276.3	3059
	App-BMao+	**0.0046**	**0.1019**	1.752	25.04	278.2	3120
	Gisma-no-Reuse	0.0075	0.1314	1.856	23.45	297.9	2625
	Gisma-no-SP	0.0088	0.1228	1.634	22.95	291.9	2588
	Gisma-no-LP	0.0069	0.0931	1.506	21.68	287.7	2498
SYN		τ=1	2	3	4	5	6
	Gisma	**0.8149**	**7.237**	**35.15**	**129.2**	**455.2**	**1259**
	Gisma-no-GS	0.8179	7.271	37.66	130.6	485.8	1311
	Gisma-no-SS	0.8162	7.254	38.67	140.3	527.4	1380
	App-BMao+	0.8201	7.305	40.90	142.8	552.3	1416
	Gisma-no-Reuse	0.8119	8.591	37.68	145.5	513.5	1403
	Gisma-no-SP	0.9334	8.348	36.77	143.7	507.3	1396
	Gisma-no-LP	0.8866	7.446	35.89	143.9	502.4	1381

Bold indicates the best (lowest) runtime; underline indicates the second-best one, among Gisma, Gisma-no-GS, Gisma-no-SS, and App-BMao+.



(a) AIDS (b) PubChem (c) Chemical1M (d) SYN

Figure 9: Effects of α on runtime performance

EXP-7. Effects of α. α is the radius threshold between `NetDAG` and `EditPathForest` in Gisma. Fig. 9 reports the impact of different α values on Gisma's performance.

8 RELATED WORK

Similarity search in doubling space. Several studies [6, 31, 36] observed that similarity search queries can be answered more efficiently in metric spaces with bounded intrinsic dimensionality. Specifically, Karger et al. [31] introduced the *expansion constant* c to measure the growth of metric spaces. Based on this constant, a sampling-based index for nearest neighbor search was designed in [31]. Similar to [31], Krauthgamer and Lee [36] proposed the *doubling constant* λ, which is defined as the minimum number of balls having radius r by which a ball with radius $2r$ can be covered. Then, they proposed the Navigating-nets index, which guarantees $(1+\epsilon)$-approximate nearest neighbor searches. Beygelzimer et al. [6] proposed cover-tree, achieving search complexity $O(c^{12} \log N)$, where N is the dataset size. Izbicki and Shelton [29] proposed parallel algorithms for constructing and searching in cover trees. Recently, Elkin and Kurlin [16] proposed a compressed version of the cover tree. Net-tree [25] organizes data into hierarchical subsets (r-nets) using the doubling constant to efficiently answer queries. Some specific properties are studied, such as *nearly doubling spaces* [22] and the *t-restricted doubling dimension* with the *net-forest* index [11].

Exact graph similarity search. The *filtering-and-verification* framework [8, 9, 32, 33, 37, 50, 54, 55] is widely adopted for graph similarity search. Here, we briefly review a few recent representative GED verification methods (*i.e.,* determining whether GED $\leq \tau$). Riesen et al. [46] proposed a practical GED computation method *A*GED* by adopting the best-first search strategy. To improve A* search for GED verification, Chang et al. [8] proposed AStar-LSa, which reduces search space by discarding dummy vertices, rearranging matching order, and providing a tighter lower bound (LSa). They subsequently introduced a tighter lower bound (BMao) in AStar-BMao [9], which is the SOTA method for GED verification. Gouda and Hassaan [23] proposed CSI-GED, which reformulates GED as common sub-structure isomorphism enumeration and also supports threshold-based verification. Kim [33] proposed Inves, an A* search method with a partition-based lower bound, and Nass [32], the SOTA for index-based GED search. However, Nass is costly to construct as its index requires $O(N^2)$ GED computations. Recently, exact GED computation methods such as DF-GED [1] (depth-first search) and the ILP-based method [14] have been proposed.

Approximate graph similarity search. Several studies address approximate graph similarity search [4, 42, 44]. Qin et al. [44] proposed GHashing, which employs hash codes and continuous embeddings as machine-learning-based filters for candidate selection, and then verifies the candidates using exact GED. LAN [42] enhances a proximity graph to query graph databases, which proposes learning-based initial point selection and neighbor pruning to reduce GED computations. In addition, machine-learning-based methods [3, 43, 45, 53] are proposed to approximate GED. Specifically, SimGNN [3] employs graph convolutional networks (GCNs) and neural tensor networks for GED prediction. Noah [53] integrates the A*-beam search with a *Graph Path Network (GPN)* heuristic. GREED [45] enables a linear-time GED prediction. GEDGNN [43] jointly predicts GED values and vertex mappings. These approaches provide practical approximations when exact GED computations do not finish. Cheng et al. [10] proposed GEDHOT, an ensemble method that combines a supervised inverse optimal transport model (GEDIOT) with an unsupervised Gromov–Wasserstein–based model (GEDGW). It reports state-of-the-art accuracy for machine-learning-based approximate GED estimation. Recently, Xu et al. [51] proposed a heuristic approximate GED computation method App-BMao, which adapts the A* search framework with an iteration limit mechanism to achieve speedup with limited accuracy loss.

9 CONCLUSION

In this paper, we analyzed the GED space and identified a two-stage phenomenon: the doubling/expansion constant is small at large distances but large at small distances. Based on this insight, we proposed Gisma, a two-stage index for approximate graph similarity search. The first stage, `NetDAG`, exploits the doubling property to narrow the search space with $O(\lambda_\alpha^{\log_2 20} \cdot \phi)$ query complexity, the lowest among existing doubling-based methods. The second stage, `EPF`, indexes shared graph edit paths for fine-grained search with three optimizations. Experiments on four benchmark datasets show that Gisma outperforms six representative methods in most cases with high recall. As future work, we plan to integrate Gisma with machine learning methods to further improve query efficiency.

AI USAGE DECLARATION

Generative AI tools were used only for polishing paper writing and assisting code writing; no research ideas were generated by AI. All AI-assisted edits were carefully reviewed and finalized by authors.

REFERENCES

[1] Zeina Abu-Aisheh, Romain Raveaux, Jean-Yves Ramel, and Patrick Martineau. 2015. An Exact Graph Edit Distance Algorithm for Solving Pattern Recognition Problems. In *Proc. of the International Conference on Pattern Recognition Applications and Methods*. 271–278.

[2] Aurelio Antelo-Collado, Ramón Carrasco-Velar, Nicolás García-Pedrajas, and Gonzalo Cerruela-García. 2020. Maximum Common Property: A New Approach for Molecular Similarity. *Journal of Cheminformatics* 12 (2020), 61. https://doi.org/10.1186/s13321-020-00462-3

[3] Yunsheng Bai, Hao Ding, Song Bian, Ting Chen, Yizhou Sun, and Wei Wang. 2019. SimGNN: A neural network approach to fast graph similarity computation. In *Proceedings of the twelfth ACM international conference on web search and data mining*. 384–392. https://doi.org/10.1145/3289600.3290967

[4] Yunsheng Bai, Hao Ding, Ken Gu, Yizhou Sun, and Wei Wang. 2020. Learning-Based Efficient Graph Similarity Computation via Multi-Scale Convolutional Set Matching.. In *Proc. of AAAI Conference on Artificial Intelligence (AAAI)*. 3219–3226.

[5] A. Patrícia Bento, Anna Gaulton, Anne Hersey, et al. 2014. The ChEMBL bioactivity database: an update. *Nucleic Acids Research* 42, D1 (2014), D1083–D1090.

[6] Alina Beygelzimer, Sham M. Kakade, and John Langford. 2006. Cover trees for nearest neighbor. In *Proceedings of the International Conference on Machine Learning (ICML)*, Vol. 148. 97–104.

[7] Silvio Cesare and Yang Xiang. 2011. Malware Variant Detection Using Similarity Search over Sets of Control Flow Graphs. In *Proc. of the International Conference on Trust, Security and Privacy in Computing and Communications*. 181–189.

[8] Lijun Chang, Xing Feng, Xuemin Lin, Lu Qin, Wenjie Zhang, and Dian Ouyang. 2020. Speeding Up GED Verification for Graph Similarity Search. In *36th IEEE International Conference on Data Engineering (ICDE)*. 793–804.

[9] Lijun Chang, Xing Feng, Kai Yao, Lu Qin, and Wenjie Zhang. 2023. Accelerating Graph Similarity Search via Efficient GED Computation. *IEEE Transactions on Knowledge and Data Engineering* 35, 5 (2023), 4485–4498. https://doi.org/10.1109/TKDE.2022.3153523

[10] Qihao Cheng, Da Yan, Tianhao Wu, Zhongyi Huang, and Qin Zhang. 2025. Computing approximate graph edit distance via optimal transport. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–26.

[11] Aruni Choudhary and Michael Kerber. 2015. Local Doubling Dimension of Point Sets. In *Proceedings of the 27th Canadian Conference on Computational Geometry (CCCG)*. Kingston, Ontario, Canada, 156–164.

[12] Kenneth L. Clarkson. 2006. Nearest-Neighbor Searching and Metric Space Dimensions. In *Nearest-Neighbor Methods for Learning and Vision: Theory and Practice*, Gregory Shakhnarovich, Trevor Darrell, and Piotr Indyk (Eds.). MIT Press.

[13] Jacobus Conradi, Anne Driemel, and Benedikt Kolbe. 2024. $(1+\varepsilon)$-ANN Data Structure for Curves via Subspaces of Bounded Doubling Dimension. *Computing in Geometry and Topology* 3, 2 (2024), 6:1–6:22. https://doi.org/10.57717/cgt.v3i2.45

[14] Andrea D'Ascenzo, Julian Meffert, Petra Mutzel, and Fabrizio Rossi. 2025. Enhancing Graph Edit Distance Computation: Stronger and Orientation-Based ILP Formulations. *Proc. VLDB Endow.* 18, 11 (2025), 4737–4749. https://doi.org/10.14778/3749646.3749726

[15] Hui Du. 2010. Chemical Molecules Search Based on Graph Similarity Measure. In *Proc. of the International Conference on Biomedical Engineering and Computer Science*. 1–4.

[16] Yury Elkin and Vitaliy Kurlin. 2023. A new near-linear time algorithm for k-nearest neighbor search using a compressed cover tree. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research)*, Vol. 202. 9267–9311.

[17] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search with the Navigating Spreading-out Graph. *Proc. VLDB Endow.* 12, 5 (2019), 461–474.

[18] Carlos Garcia-Hernandez, Alberto Fernández, and Francesc Serratosa. 2019. Ligand-Based Virtual Screening Using Graph Edit Distance as Molecular Similarity Measure. *Journal of Chemical Information and Modeling* 59, 4 (2019), 1410–1421.

[19] Siddharth Gollapudi, Ravishankar Krishnaswamy, Kirankumar Shiragur, and Harsh Wardhan. 2025. Sort Before You Prune: Improved Worst-Case Guarantees of the DiskANN Family of Graphs. In *Proceedings of the 42nd International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research)*, Vol. 267. 19787–19808.

[20] Yikai Gong, Lyu Xu, Yun Peng, Byron Choi, Jianliang Xu, and Xin Huang. 2026. Gisma: Giant-Step-Small-Step Indexing for Approximate Similarity Search in Graph Databases (Technical Report). https://github.com/McKayGONG/Gisma.

[21] Lee-Ad Gottlieb, Aryeh Kontorovich, and Robert Krauthgamer. 2014. Efficient classification for metric data. *IEEE Transactions on Information Theory* 60, 9 (2014), 5750–5759.

[22] Lee-Ad Gottlieb and Robert Krauthgamer. 2013. Proximity algorithms for nearly doubling spaces. *SIAM Journal on Discrete Mathematics* 27, 4 (2013), 1759–1769.

[23] Karam Gouda and Mosab Hassaan. 2016. CSI_GED: An efficient approach for graph edit similarity computation. In *Proc. of the IEEE International Conference on Data Engineering (ICDE)*. 265–276.

[24] Hao Guo and Youyou Lu. 2025. Achieving Low-Latency Graph-Based Vector Search via Aligning Best-First Search Algorithm with SSD. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 171–186.

[25] Sariel Har-Peled and Manor Mendel. 2005. Fast Construction of Nets in Low Dimensional Metrics, and Their Applications. In *Proceedings of the Twenty-First Annual Symposium on Computational Geometry* (Pisa, Italy) *(SCG '05)*. Association for Computing Machinery, New York, NY, USA, 150–158. https://doi.org/10.1145/1064092.1064117

[26] Jérôme Hert, Peter Willett, David J. Wilton, Pierre Acklin, Kamal Azzaoui, Edgar Jacoby, and Ansgar Schuffenhauer. 2004. Comparison of topological descriptors for similarity-based virtual screening using multiple bioactive reference structures. *Organic & Biomolecular Chemistry* 2, 22 (2004), 3256–3266.

[27] Wassily Hoeffding. 1992. A class of statistics with asymptotically normal distribution. In *Breakthroughs in statistics: Foundations and basic theory*. Springer, 308–334.

[28] Piotr Indyk and Haike Xu. 2023. Worst-case Performance of Popular Approximate Nearest Neighbor Search Implementations: Guarantees and Limitations. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36. 66239–66256. https://doi.org/10.52202/075280-2891

[29] Mike Izbicki and Christian Shelton. 2015. Faster cover trees. In *International Conference on Machine Learning*. PMLR, 1162–1170.

[30] Wenqi Jiang, Hang Hu, Torsten Hoefler, and Gustavo Alonso. 2025. Fast Graph-Based Vector Search via Hardware-Accelerated Delayed-Synchronization Traversal. *Proc. VLDB Endow.* 18, 14 (2025), 3797–3811.

[31] David R Karger and Matthias Ruhl. 2002. Finding nearest neighbors in growth-restricted metrics. In *Proceedings of the thiry-fourth annual ACM symposium on Theory of computing*. 741–750.

[32] Jongik Kim. 2021. Boosting Graph Similarity Search through Pre-Computation. In *Proc. of the ACM SIGMOD International Conference on Management of Data*. 951–963.

[33] Jongik Kim, Dong-Hoon Choi, and Chen Li. 2019. Inves: Incremental Partitioning-Based Verification for Graph Similarity Search.. In *EDBT*. 229–240.

[34] Sunghwan Kim, Jie Chen, Tiejun Cheng, Asta Gindulyte, Jia He, Siqian He, Qingliang Li, Benjamin A Shoemaker, Paul A Thiessen, Bo Yu, et al. 2023. PubChem 2023 update. *Nucleic acids research* 51, D1 (2023), D1373–D1380.

[35] Kathryn E. Kirchoff, James Wellnitz, Joshua E. Hochuli, Travis Maxfield, Konstantin I. Popov, Shawn Gomez, and Alexander Tropsha. 2024. Utilizing Low-Dimensional Molecular Embeddings for Rapid Chemical Similarity Search. In *Advances in Information Retrieval - 46th European Conference on IR Research (ECIR)*. Springer, 34–49. https://doi.org/10.1007/978-3-031-56060-6_3

[36] Robert Krauthgamer and James R. Lee. 2004. Navigating nets: simple algorithms for proximity search. In *Proceedings of the Fifteenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA, 2004*, J. Ian Munro (Ed.). SIAM, 798–807.

[37] Yongjiang Liang and Peixiang Zhao. 2017. Similarity search in graph databases: A multi-layered indexing approach. In *2017 IEEE 33rd International Conference on Data Engineering (ICDE)*. IEEE, 783–794.

[38] Y. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836.

[39] David Mendez, Anna Gaulton, A. Patrícia Bento, Jon Chambers, Marleen De Veij, Eloy Félix, María Paula Magariños, Juan F. Mosquera, Prudence Mutowo, Michał Nowotka, et al. 2019. ChEMBL: towards direct deposition of bioassay data. *Nucleic Acids Research* 47, D1 (2019), D930–D940.

[40] Misagh Naderi, Chris Alvin, Yun Ding, Supratik Mukhopadhyay, and Michal Brylinski. 2016. A graph-based approach to construct target-focused libraries for virtual screening. *Journal of cheminformatics* 8, 1 (2016), 1–16.

[41] NextMove Software. 2026. SmallWorld Search. https://sw.docking.org. Accessed: July 2026.

[42] Yun Peng, Byron Choi, Tsz Nam Chan, and Jianliang Xu. 2022. Lan: Learning-based approximate k-nearest neighbor search in graph databases. In *2022 IEEE 38th international conference on data engineering (ICDE)*. IEEE, 2508–2521.

[43] Chengzhi Piao, Tingyang Xu, Xiangguo Sun, Yu Rong, Kangfei Zhao, and Hong Cheng. 2023. Computing Graph Edit Distance via Neural Graph Matching. *Proc. VLDB Endow.* 16, 8 (2023), 1817–1829.

[44] Zongyue Qin, Yunsheng Bai, and Yizhou Sun. 2020. GHashing: Semantic Graph Hashing for Approximate Similarity Search in Graph Databases. In *Proc. of the ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2062–2072.

[45] Rishabh Ranjan, Siddharth Grover, Sourav Medya, Venkatesan Chakaravarthy, Yogish Sabharwal, and Sayan Ranu. 2022. Greed: A neural framework for learning graph distance functions. *Advances in Neural Information Processing Systems* 35 (2022), 22518–22530.

[46] Kaspar Riesen, Stefan Fankhauser, and Horst Bunke. 2007. Speeding up graph edit distance computation with a bipartite heuristic.. In *MLG*. Citeseer, 21–24.

[47] Roger Sayle, John May, Noel O'Boyle, Andrew Grant, Stefan Senger, and Darren Green. 2016. Chemical Similarity based on Graph Edit Distance: Efficient Implementation and the Challenges of Evaluation. 7th Sheffield Conference on Chemoinformatics, Sheffield, UK.

[48] Benjamin I. Tingle, Khanh G. Tang, Mar Castanon, John J. Gutierrez, Munkhzul Khurelbaatar, Chinzorig Dandarchuluun, Yurii S. Moroz, and John J. Irwin. 2023. ZINC-22 – A Free Multi-Billion-Scale Database of Tangible Compounds for Ligand Discovery. *Journal of Chemical Information and Modeling* 63, 4 (2023), 1166–1176. https://doi.org/10.1021/acs.jcim.2c01253

[49] Runzhong Wang, Tianqi Zhang, Tianshu Yu, Junchi Yan, and Xiaokang Yang. 2021. Combinatorial learning of graph edit distance via dynamic embedding. In *Proc. of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 5241–5250.

[50] Xiaoli Wang, Xiaofeng Ding, Anthony K.H. Tung, Shanshan Ying, and Hai Jin. 2012. An efficient graph indexing method. In *Proc. of the IEEE International Conference on Data Engineering*. IEEE, 210–221.

[51] Mouyi Xu and Lijun Chang. 2025. Graph Edit Distance Estimation: A New Heuristic and A Holistic Evaluation of Learning-based Methods. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–24.

[52] Xiaoliang Xu, Mengzhao Wang, Yuxiang Wang, and Dingcheng Ma. 2021. Two-stage routing with optimized guided search and greedy algorithm on proximity graph. *Knowledge-Based Systems* 229 (2021), 107305. https://doi.org/10.1016/j.knosys.2021.107305

[53] Lei Yang and Lei Zou. 2021. Noah: Neural-optimized A* Search Algorithm for Graph Edit Distance Computation. In *Proc. of IEEE International Conference on Data Engineering (ICDE)*. 576–587.

[54] Zhiping Zeng, Anthony K. H. Tung, Jianyong Wang, Jianhua Feng, and Lizhu Zhou. 2009. Comparing Stars: On Approximating Graph Edit Distance. *Proc. VLDB Endow.* 2, 1 (2009), 25–36.

[55] Xiang Zhao, Chuan Xiao, Xuemin Lin, Wei Wang, and Yoshiharu Ishikawa. 2013. Efficient processing of graph similarity queries with edit distance constraints. *VLDB J.* 22, 6 (2013), 727–752.

[56] Xiang Zhao, Chuan Xiao, Xuemin Lin, Wenjie Zhang, and Yang Wang. 2018. Efficient structure similarity searches: a partition-based approach. *VLDB J.* 27, 1 (2018), 53–78.

[57] Weiguo Zheng, Lei Zou, Xiang Lian, Dong Wang, and Dongyan Zhao. 2013. Graph Similarity Search with Edit Distance Constraint in Large Graph Databases. In *Proceedings of ACM International Conference on Information & Knowledge Management (CIKM)*. 1595–1600.

[58] Yuanyuan Zhu, Lu Qin, Jeffrey Xu Yu, and Hong Cheng. 2020. Answering Top-k Graph Similarity Queries in Graph Databases. *IEEE Transactions on Knowledge and Data Engineering* 32, 8 (2020), 1459–1474.

A APPENDIX

A.1 Supplementary Analysis of NetDAG

This subsection establishes the space, query time, and construction complexity of NetDAG. We adopt a lemma from [21] (Lemma A.1) to bound the number of children and parents per node (Lemmas A.2–A.3) and the number of inter-layer edges per pair (Lemma A.4). These lead to the main complexity results: space (Theorem A.5), query time (Theorem A.7), and construction time (Theorem A.8). We also express the bounds in terms of the expansion constant c (Lemma A.9) for comparison with Cover Tree.

Lemma A.1 ([21]). *Let (X, d) be a metric space with doubling constant λ. Suppose $S \subset X$ is finite, with minimum inter-point distance at least $r > 0$ and diameter at most* diam > 0. *Then*

$$|S| \;\le\; \lambda^{\log_2\left(\frac{2\,\mathrm{diam}}{r}\right)}.$$

Corollary 1 (Number of r-net points in a ball)**.** Given an r-net Y in a metric space (X, d) with doubling constant λ, for any ball B of radius at most R, the number of points of Y contained in B is at most

$$\lambda^{\log_2\left(\frac{4R}{r}\right)}.$$

Proof of Corollary 1. Let $Y' = Y \cap B$. Because Y' is contained in a ball of radius R, the diameter of Y' is at most $2R$. Meanwhile, Y being an r-net implies any two distinct points in Y' are at least r apart. Hence Y' meets the conditions of Lemma A.1 with minimum inter-point distance r and diameter at most $2R$. Applying Lemma A.1 yields

$$|Y'| \;=\; |\,Y \cap B| \;\le\; \lambda^{\log_2\left(\frac{2(2R)}{r}\right)} \;=\; \lambda^{\log_2\left(\frac{4R}{r}\right)}.$$

□

Lemma A.2 (Upper bound on number of children). *In an* NetDAG *index (Def. 4.1) with doubling constant λ_α down to α, the ball radius of centers in layer Y_0 and $0 \le \tau \le \alpha$, the number of children of any node is at most*

$$\lambda_\alpha^{\log_2 20}.$$

Proof. To obtain this constant, we analyze the child range and separation of NetDAG. From Def. 4.1, each child p_x^i of p_j^{i+1} satisfies $d\big(p_j^{i+1},\ p_x^i\big) \le 3\,\alpha\,2^i + 2\,\tau$, so all children are in the ball $B\big(p_j^{i+1},\ 3\,\alpha\,2^i + 2\,\tau\big)$. Since Y_i is an $(\alpha\,2^i)$-net, the minimal distance between any two distinct nodes in Y_i is at least $\alpha\,2^i$. Applying Corollary 1 with $R = 3\,\alpha\,2^i + 2\,\tau$ and $r = \alpha\,2^i$ gives an upper bound of

$$\lambda^{\log_2\left(\frac{4\,(3\,\alpha\,2^i+2\,\tau)}{\alpha\,2^i}\right)} = \lambda^{\log_2\left(12+\frac{8\,\tau}{\alpha\,2^i}\right)}.$$

Since $0 \le \tau \le \alpha$ and children are always in a layer Y_i with $i \ge 0$, we have $12 + \frac{8\tau}{\alpha 2^i} \le 12 + \frac{8\alpha}{\alpha 2^i} \le 20$. Replacing the doubling constant by λ_α for the relevant scale yields the claimed bound $\lambda_\alpha^{\log_2 20}$. □

Lemma A.3 (Upper bound on number of parents). *In an* NetDAG *index (Def. 4.1) with doubling constant λ_α down to α, the ball radius of centers in layer Y_0 and $0 \le \tau \le \alpha$, the number of parents of any node p_x^i is at most*

$$\lambda_\alpha^{\log_2 10}.$$

Proof. A node p_x^i in layer Y_i can only have parents in layer Y_{i+1} that satisfy $d\big(p_x^i,\ p_j^{i+1}\big) \le 3\,\alpha\,2^i + 2\,\tau$, so all possible parents are in $B\big(p_x^i,\ 3\,\alpha\,2^i + 2\,\tau\big) \cap Y_{i+1}$. Since Y_{i+1} is an $(\alpha\,2^{\,i+1})$-net, any two distinct points there are at least $\alpha\,2^{\,i+1}$ apart. Applying Corollary 1 with $R = 3\,\alpha\,2^i + 2\,\tau$ and $r = \alpha\,2^{\,i+1}$ gives an upper bound of

$$\lambda^{\log_2\left(\frac{4\,(3\,\alpha\,2^i+2\,\tau)}{\alpha\,2^{i+1}}\right)} = \lambda^{\log_2\left(6+\frac{4\,\tau}{\alpha\,2^i}\right)}.$$

Since $0 \le \tau \le \alpha$ and $i \ge 0$, we have $6 + \frac{4\tau}{\alpha 2^i} \le 6 + \frac{4\alpha}{\alpha\cdot 1} = 10$. Replacing the doubling constant by λ_α for the relevant scale yields the claimed bound $\lambda_\alpha^{\log_2 10}$. □

In a NetDAG, the same data point p_k may appear as a child of p_j in multiple consecutive layers. The following lemma shows that the number of such layers is bounded by a constant, which is used in the space complexity proof (Theorem A.5).

For each data point p_x, let $i_x^{\max}$ be such that $Y_{i_x^{\max}}$ is the highest layer containing p_x (*i.e.,* $p_x \in Y_{i_x^{\max}}$ but $p_x \notin Y_{i_x^{\max}+1}$).

Lemma A.4. *For any pair of data points (p_j, p_k) in an* NetDAG *index, if the parent-child edge $(p_j^{i_0+1}, p_k^{i_0})$ exists at some layer i_0, then:* ***(a)*** *the edge exists at all higher layers as long as p_j and p_k exist: the edge (p_j^{i+1}, p_k^i) exists at all layers i with $i_0 \le i \le \min(i_j^{\max} - 1,\ i_k^{\max})$;* ***(b)*** *the number of such layers is at most a constant:* $\min(i_j^{\max} - 1,\ i_k^{\max}) - i_0 + 1 \le \lfloor \log_2(2\tau/\alpha + 3) \rfloor + 1$.

Proof of Lemma A.4. By Def. 4.1, the edge (p_j^{i+1}, p_k^i) exists at layer i if and only if $d(p_j, p_k) \le 3 \cdot \alpha \cdot 2^i + 2\tau$, $p_j \in Y_{i+1}$, and $p_k \in Y_i$.

(a) Since $(p_j^{i_0+1}, p_k^{i_0})$ exists, we have $d(p_j, p_k) \le 3\alpha \cdot 2^{i_0} + 2\tau$. For all $i \ge i_0$, $3\alpha \cdot 2^{i_0} + 2\tau \le 3\alpha \cdot 2^i + 2\tau$, so the distance condition is satisfied. The edge also requires $p_k \in Y_i$ ($i \le i_k^{\max}$) and $p_j \in Y_{i+1}$ ($i \le i_j^{\max} - 1$). Hence the edge exists at all layers i with $i_0 \le i \le \min(i_j^{\max} - 1,\ i_k^{\max})$.

(b) By (a), the edge $(p_j^{i_0+m+1}, p_k^{i_0+m})$ exists, where $m = \min(i_j^{\max} - 1,\ i_k^{\max}) - i_0$. Since $Y_{i_0+m+1} \subseteq Y_{i_0+m}$, both $p_j, p_k \in Y_{i_0+m}$. Then by the $(\alpha \cdot 2^{i_0+m})$-net property (Def. 2.4), $d(p_j, p_k) \ge \alpha \cdot 2^{i_0+m}$. Combined with $d(p_j, p_k) \le 3\alpha \cdot 2^{i_0} + 2\tau$ since the edge $(p_j^{i_0+1}, p_k^{i_0})$ exists:

$$\alpha \cdot 2^{i_0+m} \le d(p_j, p_k) \le 3 \cdot \alpha \cdot 2^{i_0} + 2\tau.$$

Dividing by $\alpha \cdot 2^{i_0}$ gives $2^m \le 3 + 2\,\tau/(\alpha \cdot 2^{i_0}) \le 3 + 2\,\tau/\alpha$ (since $i_0 \ge 0$), so $m \le \lfloor \log_2(2\tau/\alpha + 3) \rfloor$, and therefore $m + 1 = \min(i_j^{\max} - 1,\ i_k^{\max}) - i_0 + 1 \le \lfloor \log_2(2\tau/\alpha + 3) \rfloor + 1$. □

Theorem A.5 (Space Complexity of NetDAG). *The total space of* NetDAG *is $O(\lambda_\alpha^{O(1)} N) = O(N)$, where N is the database size and λ_α is the doubling constant down to α, the ball radius of centers in layer Y_0.*

Proof. The space of NetDAG equals the number of nodes and edges. By Lemma A.2, each node has at most $\lambda_\alpha^{\log_2 20}$ children. It therefore suffices to show that the total number of nodes is $O(N)$.

We extend the node-counting approach of [36] to NetDAG using the marking-set argument below, counting only nodes with multiple children. By Def. 4.1, $Y_i \subseteq Y_{i-1}$, every node p_j^i is by default its own child p_j^{i-1} in the next layer. Thus a node with exactly one child need not be stored and does not contribute to NetDAG space. It remains to show that the number of nodes with multiple children is $O(N)$.

For ease of analysis of counting those nodes, we define a *marking set* for each node p_j^i that has multiple children:

$$M_j^i = \left\{ p_k \;\middle|\; \Psi_1(p_j^i,\, p_k^{i-1}) \;\vee\; \Psi_2(p_k^{i+1},\, p_j^i) \right\},$$

where Ψ_1 and Ψ_2 are two *marking conditions*:
$\Psi_1(p_j^i,\, p_k^{i-1})$: the edge $(p_j^i,\, p_k^{i-1})$ exists and $i-1 \in [i_k^{\max}-m,\; i_k^{\max}]$;
$\Psi_2(p_k^{i+1},\, p_j^i)$: the edge $(p_k^{i+1},\, p_j^i)$ exists and $i \in [i_j^{\max}-m,\; i_j^{\max}]$;
and $m = \lfloor \log_2(2\,(\tau/\alpha)+3) \rfloor$.

Since each node with multiple children has only one marking set, the number of such nodes equals the number of marking sets. We show that every M_j^i is non-empty (Step 1), so the number of marking sets is at most $\sum_{j,i} |M_j^i|$, where $j \in \{1,\dots,N\}$, $i \in \{0,\dots,\phi\}$, and the sum is over all nodes p_j^i with multiple children. We then show $\sum_{j,i} |M_j^i| = O(N)$ (Step 2). Together, these two steps prove that the number of nodes with multiple children is $O(N)$.

Step 1: Every marking set is non-empty. Consider an arbitrary node $p_j^{i_0}$ with multiple children, and a child $p_k^{i_0-1}$ with $k \neq j$. By the definition of $M_j^{i_0}$, it suffices to show that $\Psi_1(p_j^{i_0},\, p_k^{i_0-1})$ or $\Psi_2(p_k^{i_0+1},\, p_j^{i_0})$ holds. Since the edge $(p_j^{i_0},\, p_k^{i_0-1})$ exists, by Lemma A.4:

Case 1: $i_k^{\max} < i_j^{\max}$. By (b), $i_k^{\max} - (i_0-1) \le m$, therefore $i_0 - 1 \in [i_k^{\max}-m,\; i_k^{\max}]$, so $\Psi_1(p_j^{i_0},\, p_k^{i_0-1})$ holds. Hence $p_k \in M_j^{i_0}$.

Case 2: $i_k^{\max} \ge i_j^{\max}$.

- If $i_k^{\max} = i_0$: $i_0 - 1 \in [i_k^{\max}-m,\; i_k^{\max}]$ (since $m \ge 1$), so $\Psi_1(p_j^{i_0},\, p_k^{i_0-1})$ holds. Hence $p_k \in M_j^{i_0}$.
- If $i_k^{\max} \ge i_0+1$: $p_k \in Y_{i_0+1}$, and $d(p_j, p_k) \le 3\alpha\cdot 2^{i_0-1} + 2\tau \le 3\alpha\cdot 2^{i_0} + 2\tau$, so the edge $(p_k^{i_0+1},\, p_j^{i_0})$ exists. By (b), $\min(i_j^{\max}-1,\; i_k^{\max}) = i_j^{\max}-1$, so $(i_j^{\max}-1) - (i_0-1) \le m$, *i.e.*, $i_0 \in [i_j^{\max}-m,\; i_j^{\max}]$, so $\Psi_2(p_k^{i_0+1},\, p_j^{i_0})$ holds. Hence $p_k \in M_j^{i_0}$.

In both cases, $|M_j^{i_0}| \ge 1$.

Step 2: Upper bound of the sum of sizes of all marking sets. Every element of M_j^i has a parent-child edge satisfying Ψ_1 or Ψ_2. Each such edge contributes at most 2 elements. For each data point p_b as the child in an edge, the condition $i \in [i_b^{\max}-m,\; i_b^{\max}]$ (used in both Ψ_1 and Ψ_2) holds for at most $m+1$ layers ($m+1 = \lfloor \log_2(2\,(\tau/\alpha)+3) \rfloor + 1$ is a constant), and at each such layer i, the number of parents p_a^{i+1} is $O(1)$ by Lemma A.3. Hence each p_b is involved in $O(1)$ edges satisfying Ψ_1 or Ψ_2, and $\sum_{j,i} |M_j^i| \le N \cdot (m+1) \cdot O(1) \cdot 2 = O(N)$.

Putting these together, since each marking set is non-empty, the number of nodes with multiple children is at most $\sum_{j,i} |M_j^i| = O(N)$. By Lemma A.2, each node has only a constant number of children. Thus the total space of NetDAG is $O(N)$. □

Lemma A.6. *Given a* NetDAG*, for any node* $p^{i+1} \in Y_{i+1}$, $B(p^{i+1},\, \alpha\cdot 2^{i+1} + 2\tau) \subseteq \bigcup_{p^i \in \text{children}(p^{i+1})} B(p^i,\, \alpha\cdot 2^i)$.

Proof. Let x be an arbitrary point in $B(p^{i+1}, \alpha\cdot 2^{i+1}+2\tau)$. Since Y_i is an $(\alpha\cdot 2^i)$-net of $\mathcal{D}$, there exists $p^i \in Y_i$ such that $d(x, p^i) \le \alpha\cdot 2^i$, and thus $x \in B(p^i, \alpha\cdot 2^i)$. Furthermore, $d(p^{i+1}, p^i) \le d(p^{i+1}, x) + d(x, p^i) \le (\alpha\cdot 2^{i+1} + 2\tau) + \alpha\cdot 2^i = 3\alpha\cdot 2^i + 2\tau$. By Def. 4.1, this implies $(p^{i+1}, p^i) \in E$, so $p^i \in \text{children}(p^{i+1})$. □

Corollary 2. Given a NetDAG, if $\mathcal{A} \neq \emptyset$ and $\mathcal{A} \subseteq B(p^{i+1},\, \alpha\cdot 2^{i+1} + 2\tau)$ for some $p^{i+1} \in Y_{i+1}$, then there exists a child $p^i \in \text{children}(p^{i+1})$ such that $d(p^i, Q) \le \alpha\cdot 2^i + \tau$.

Theorem A.7 (Time Complexity of Querying NetDAG). *Given a* NetDAG *with ϕ layers, Alg. 1 completes in $O(\lambda_\alpha^{\log_2 20}\cdot\phi)$ distance computations (NDCs).*

Proof. The algorithm traverses from layer Y_ϕ down to the leaf layer Y_1, maintaining a single pivot at each layer. At each layer i (for $i = \phi - 1$ down to 1), we examine all children of the current pivot p^{i+1} to find one satisfying

$$d(p_j^i, Q) \le \alpha\cdot 2^i + \tau.$$

By Lemma A.2, any node has at most

$$\lambda_\alpha^{\log_2\left(12+\frac{8\tau}{\alpha\cdot 2^i}\right)} \;\le\; \lambda_\alpha^{\log_2 20}$$

children. Therefore, the algorithm performs at most $\lambda_\alpha^{\log_2 20}$ distance computations per layer across ϕ layers. This yields a total of

$$O(\lambda_\alpha^{\log_2 20}\cdot\phi)$$

distance computations.

Finally, note that $\phi = O(\log_2(\Delta/\alpha))$, where Δ is the dataset diameter (the maximum pairwise distance between graphs). □

Theorem A.8 (Construction Complexity of NetDAG). *The* NetDAG *index can be constructed using $O(\lambda_\alpha^{O(1)} N\phi)$ distance computations, where N is the database size, ϕ is the number of layers, and λ_α is the doubling constant down to radius α.*

Proof. Section 3.1 of [25] shows that the number of distance computations conducted during the greedy-permutation construction is $O(\lambda^{O(1)} Ns)$, where s is the number of *phases*. The radius sequence $r_1 \ge r_2 \ge \cdots$ is non-increasing, and a *phase* is the maximal interval of consecutive iterations in which the greedy radius r_k stays within $(r/2, r]$ for some value r. Within any such *phase*, each point is scanned only $\lambda^{O(1)}$ times (Section 3.1 in [25]), because the doubling property bounds the size of every *friends list* and hence, the number of times a point recomputes its distance to the newly selected center. Summed over all *phases*, the total number of distance computations is therefore $O(\lambda^{O(1)} Ns)$.

In our setting, the greedy radii decrease from $\alpha\cdot 2^\phi$ to α, and the radius changes by at most a factor of two between consecutive layers. Hence the construction encounters exactly $\phi + 1$ distinct radii, which correspond directly to the $\phi + 1$ layers of NetDAG. Substituting $s = \phi + 1$ gives

$$O(\lambda^{O(1)} Ns) = O(\lambda^{O(1)} N\phi).$$

Finally, all radii used in our construction satisfy $r \ge \alpha$, so all arguments in [25] remain valid when the global doubling constant λ is replaced by the doubling constant down to α, $\lambda_\alpha := \max_{p\in\mathcal{X}, r\ge\alpha} \lambda(p, r)$. Thus the total number of distance computations becomes $O(\lambda_\alpha^{O(1)} N\phi)$. □

A.1.1 Time and space complexity in terms of expansion constant.

The preceding theorems express all bounds using the doubling constant λ. The time and space complexity of NetDAG can also be stated in terms of the expansion constant c, which allows direct comparison with Cover Tree [6] (Tab. 2). We adopt and generalize a technique from Cover Trees [6] as follows.

Lemma A.9. *Given a metric space $(\mathcal{X}, d)$ with expansion constant c (Def. 2.2) and an r-separated set $S_r \subseteq \mathcal{X}$, for any point $p \in \mathcal{X}$ and*

any integer $k \geq 0$, the number of points of S_r contained in the ball $B(p, 2^k r)$ is at most c^{k+3}.

Proof. The proof generalizes the *width bound* (Lemma 4.1) of Cover Trees [6] to arbitrary k. (Substituting $r = 2^{i-1}$ and $k = 1$ recovers the Cover Tree bound c^4 exactly; our formulation yields the c^{k+3} bounds used in Tab. 2.)

We fix $p \in \mathcal{X}$ and let $S := S_r \cap B(p, 2^k r)$. Since S_r is an r-separated set, any two points of S are at least r apart. Hence, the balls $\{B(p', r/2) : p' \in S\}$ are pairwise disjoint. Moreover, for any $p' \in S$,

$$B(p', r/2) \subseteq B\big(p,\ 2^k r + r/2\big) \subseteq B\big(p,\ 2^{k+1} r\big),$$

so

$$\sum_{p' \in S} |B(p', r/2)| \ \leq\ \big|B\big(p, 2^{k+1} r\big)\big|. \tag{3}$$

Let $p^* \in \arg\min_{p' \in S} |B(p', r/2)|$. Since $d(p, p^*) \leq 2^k r$, for any $y \in B\big(p, 2^{k+1} r\big)$,

$$d(y, p^*) \leq d(y, p) + d(p, p^*) \leq 2^{k+1} r + 2^k r < 2^{k+2} r.$$

Hence,

$$B\big(p, 2^{k+1} r\big) \subset B\big(p^*, 2^{k+2} r\big).$$

Since $B(p, 2^{k+1} r) \subset B(p^*, 2^{k+2} r)$, we have $\big|B\big(p, 2^{k+1} r\big)\big| \leq \big|B\big(p^*, 2^{k+2} r\big)\big|$. By the expansion inequality $|B(p, 2r)| \leq c\,|B(p, r)|$ (Def. 2.2),

$$\begin{aligned} \big|B\big(p^*, 2^{k+2} r\big)\big| \ &\leq\ c\,\big|B\big(p^*, 2^{k+1} r\big)\big| \ \leq\ c^2\,\big|B\big(p^*, 2^k r\big)\big| \\ &\leq\ \cdots \ \leq\ \boldsymbol{c^{k+3}}\,\big|B\big(p^*, r/2\big)\big|. \end{aligned} \tag{4}$$

Combining (3) and (4), since $|B(p^*, r/2)| \leq |B(p', r/2)|$ for all $p' \in S$,

$$|S| \cdot |B(p^*, r/2)| \ \leq\ \sum_{p' \in S} |B(p', r/2)| \ \leq\ \boldsymbol{c^{k+3}}\,|B(p^*, r/2)|,$$

so, $|S| \leq \boldsymbol{c^{k+3}}$. □

Similarly to Lemmas A.2–A.3 and Theorems A.5–A.8, using Lemma A.9, all complexity bounds of NetDAG can be restated in terms of the expansion constant c_α. Specifically, the number of children per node becomes c_α^6 (applying Lemma A.9 with $r = \alpha \cdot 2^i$ and $k = \lceil \log_2(3 + 2\tau/(\alpha \cdot 2^i)) \rceil \leq 3$), giving a query time complexity of $O(c_\alpha^6 \cdot \phi)$ and a construction time complexity of $O(c_\alpha^{O(1)} N\phi)$. The space complexity remains $O(N)$. These bounds are summarized in Tab. 2.

A.1.2 Full derivations of the analysis of NetDAG *and other doubling-based indexes.* We compare NetDAG with other representative doubling-based indexes for non-graph data [6, 25, 36] and summarize the results in Tab. 9, which is the full version of Tab. 2 of the main text.

Since NetDAG serves as the first stage of Gisma and narrows down the query answers to several α-balls in EditPathForest, we compare these methods under the same setting: all methods build a hierarchical structure above the same α-net Y_0 and traverse top-down to find the candidate α-balls in Y_0. For fair comparison, the ratio of the radius of the parent layer to its child layer is $\kappa = 2$, for all methods. Thus, the radius of layer i is $r_i = \alpha \cdot 2^i$. All methods have the same number of layers ϕ, and r_i is written as r in Tab. 2 since the table holds for all $1 \leq i \leq \phi - 1$. For methods that give only asymptotic bounds of the form $\lambda^{O(1)}$, we derive specific constants using Lemma A.1 and Corollary 1 in Appx. A.1.2.

The efficiency of the traversal is thus determined by the per-layer NDC. At each layer, the traversal maintains a candidate set and expands the children of each candidate, so the per-layer NDC is the product of the *candidate set size* and the *number of children per node*. In a doubling space, these two values are bounded by their range (ball radius containing the set) and separation (minimum pairwise distance) respectively, as shown in Tab. 2. In NetDAG, the child range $3r + 2\tau$ includes a 2τ margin that guarantees the coverage of the answer ball $B(Q, \tau)$, so that at least one child satisfies $d(p, Q) \leq r + \tau$ (Corollary 2). This allows maintaining only a single pivot rather than a candidate set (Alg. 1), reducing the per-layer NDC to $\lambda_\alpha^{\log_2 20}$.

We derive the entries in Tab. 2. Throughout, r denotes r_i and the parent-layer radius is $2r$ ($\kappa = 2$).

NetDAG. We analyze the query time complexity of NetDAG in terms of both λ and c, for comparison with methods that use either measure of dimensionality. In terms of λ, the number of children is $\lambda_\alpha^{\log_2 20}$ (Lemma A.2). In terms of c, the child range is $3r + 2\tau$ and the separation is r. By Lemma A.9, the number of children is at most $c_\alpha^{\lceil \log_2(3 + 2\tau/r) \rceil + 3} \leq c_\alpha^6$. Since Alg. 1 maintains a single pivot (candidate set of size 1), the per-layer NDC equals the number of children: $\lambda_\alpha^{\log_2 20}$ (or c_α^6). At each layer, the pivot satisfies $d(p, Q) \leq r + \tau$ (Theorem 4.2).

Cover Tree [6]. Cover Tree uses the expansion constant c. By the covering and separation properties (Section 2 of [6]), the child range is $2r$ and the child separation is r. The number of children is c^4 (Lemma 4.1 of [6]). The candidate set size is c^5 (Theorem 5 of [6]). The per-layer NDC is $c^4 \cdot c^5 = c^9$. Let a be the nearest neighbor of Q in $\mathcal{D}$, *i.e.*, $d(a, Q) \leq \tau$. By induction on the covering property (Section 2 of [6]), the candidate set at each layer contains a point within $2r$ of a. By the triangle inequality, $d(\text{cand}, Q) \leq d(\text{cand}, a) + d(a, Q) \leq 2r + \tau$. Since all radii satisfy $r \geq \alpha$, c can be replaced by c_α.

Navigating Nets [36]. Navigating Nets uses the doubling constant λ. Let a be the nearest neighbor of Q in $\mathcal{D}$. The child range is $\gamma R = 8r$ ($\gamma = 4$, $R = 2r$; Section 2.1 of [36]) and the child separation is r (Lemma 2.1 of [36]). By Corollary 1, each node has at most $\lambda^{\log_2(4 \cdot 8r/r)} = \lambda^5$ children. For the candidate set, let Z_{2r} denote the candidate set maintained at the parent scale $R = 2r$. Lemma 2.4 of [36] gives $d(a, z) \leq 2R = 4r$ for some $z \in Z_{2r}$, so $d(Q, Z_{2r}) \leq d(Q, a) + 4r$. The filter retains candidates within $d(Q, Z_{2r}) + 2r$ of Q (Procedure Approx-NNS of [36]), giving a candidate range of at most $d(Q, a) + 6r \leq 7r$ (since $d(Q, a) \leq \tau \leq \alpha \leq r$). By Corollary 1 with separation r, the candidate set size is at most $\lambda^{\log_2(4 \cdot 7r/r)} = \lambda^{\log_2 28}$. The per-layer NDC is $\lambda^5 \cdot \lambda^{\log_2 28} = \lambda^{5 + \log_2 28}$. At each layer, Lemma 2.4 of [36] at scale r gives $d(a, z) \leq 2r$, so $d(z, Q) \leq 2r + \tau$. Since all radii satisfy $r \geq \alpha$, λ can be replaced by λ_α.

Net-Tree [25]. Net-Tree uses the doubling constant λ. Let a be the nearest neighbor of Q in $\mathcal{D}$. The child range is $4r$ (Lemma 3.9(ii) of [25]) and the child separation is r (Lemma 3.9(iii) of [25]). By Corollary 1, each node has at most $\lambda^{\log_2(4 \cdot 4r/r)} = \lambda^{\log_2 16} = \lambda^4$ children. The candidate set has range $13r$ and separation r (Section 3.4 and Lemma 3.9(iii) of [25]). By Corollary 1, the candidate set size is at most $\lambda^{\log_2(4 \cdot 13r/r)} = \lambda^{\log_2 52}$. The per-layer NDC is $\lambda^4 \cdot \lambda^{\log_2 52} = \lambda^{4 + \log_2 52}$. By the covering property (Lemma 3.9(iv)

Table 9: Full comparison of NetDAG and representative doubling-based methods ($\kappa = 2$)

Index	Child range	Child separation	Max. # children	Traversal proceeds to	Cand. range	Cand. separation	Max. cand. set size	Dist. to Q	O(NDC/layer)
NetDAG	$3\mathbf{r} + 2\tau$	$\mathbf{r}$	$\lambda_\alpha^{\log_2 20}$ (or c_α^6)	ONE child	N/A	N/A	1	$\le \mathbf{r} + \tau$	$\lambda_\alpha^{\log_2 20}$ (or c_α^6)
Net-Tree [25]	$4r$	r	λ_α^4	child set	$13r$	r	$\lambda_\alpha^{\log_2 52}$	$\le 4r + \tau$	$\lambda_\alpha^{4+\log_2 52}$
Navigating Nets [36]	$8r$	r	λ_α^5	child set	$d_{\min}(Q,\ \text{cand\_set}) + 2r$	r	$\lambda_\alpha^{\log_2 28}$	$\le 2r + \tau$	$\lambda_\alpha^{5+\log_2 28}$
Cover Tree [6]	$2r$	r	c_α^4	child set	$d_{\min}(Q,\ \text{cand\_set}) + 2r$	r	c_α^5	$\le 2r + \tau$	c_α^9

Algorithm 4: NetDAG Construction Algorithm

Input: $\mathcal{D}$: data points; $\alpha > 0$; $0 \le \tau \le \alpha$.
Output: Layers $Y_0, \dots, Y_\phi$; edge set E (within NetDAG) and E' (between Y_1 and Y_0).

// Initialization (same as greedy permutation)
1 $E \leftarrow \emptyset$;
2 $E' \leftarrow \emptyset$;
3 choose any $p_0 \in \mathcal{D}$ as the first center;
4 mark p_0 as selected;
5 $\delta(p_0) \leftarrow 0$;
6 $H \leftarrow$ empty max-heap with key $= \delta(x)$;
7 **foreach** $x \in \mathcal{D}$ with $x \neq p_0$ **do**
8 $\quad \delta(x) \leftarrow d(x, p_0)$;
9 $\quad$ Center$(x) \leftarrow p_0$;
10 $\quad$ insert x into ServedBy(p_0);
11 $\quad$ push(x, H);
12 $k \leftarrow 1$;
13 $r \leftarrow +\infty$;
14 **while** $r > \alpha$ **do**
$\quad$ // Select new center and determine layers (same as greedy permutation)
15 $\quad u \leftarrow \text{pop}(H)$; $k \leftarrow k + 1$; $p_k \leftarrow u$; $r \leftarrow \delta(u)$;
16 $\quad$ mark p_k as selected;
17 $\quad$ **if** $k = 2$ **then**
18 $\quad\quad \phi \leftarrow \lceil \log_2(r/\alpha) \rceil$;
19 $\quad\quad Y_\phi \leftarrow \{p_0\}$;
20 $\quad\quad j \leftarrow \phi - 1$;
$\quad$ // Create layers and assign edges (our modification)
21 $\quad$ **if** $k \ge 2$ and $j \ge 1$ and $r \le \alpha \cdot 2^j$ **then**
22 $\quad\quad i_{\min} \leftarrow \max\left(1,\ \lceil \log_2(r/\alpha) \rceil\right)$;
23 $\quad\quad$ **for** $i = j$ to $i_{\min}$ **do**
24 $\quad\quad\quad Y_i \leftarrow \{p_0, \dots, p_{k-1}\}$;
25 $\quad\quad\quad$ **foreach** $p_j^{i+1} \in Y_{i+1}$ **do**
26 $\quad\quad\quad\quad$ **foreach** $p_\ell^i \in$ Friends(p_j^{i+1}) with $p_\ell^i \in Y_i$ **do**
27 $\quad\quad\quad\quad\quad$ **if** $d(p_\ell^i, p_j^{i+1}) \le 3\alpha \cdot 2^i + 2\tau$ **then**
28 $\quad\quad\quad\quad\quad\quad$ add (p_j^{i+1}, p_ℓ^i) to E;
29 $\quad\quad j \leftarrow i_{\min} - 1$;
$\quad$ // Update friends and served points (same as greedy permutation)
30 $\quad$ update all affected Friends$(\cdot)$ and ServedBy$(\cdot)$ using Alg. 5;
31 $Y_0 \leftarrow \{p_0, \dots, p_{k-1}\}$;
// The same pass continues one more step: connect Y_1 to the α-net Y_0
32 **foreach** $p_j^1 \in Y_1$ **do**
33 $\quad$ **foreach** $p_\ell^0 \in$ Friends(p_j^1) with $p_\ell^0 \in Y_0$ **do**
34 $\quad\quad$ **if** $d(p_\ell^0, p_j^1) \le 3\alpha + 2\tau$ **then**
35 $\quad\quad\quad$ add (p_j^1, p_ℓ^0) to E';
36 **return** $(Y_0, \dots, Y_\phi, E, E')$;

of [25]), $d(a, \text{cand}) \le 4r$. By the triangle inequality, $d(\text{cand}, Q) \le 4r + \tau$. Since all radii satisfy $r \ge \alpha$, λ can be replaced by λ_α.

A.1.3 Detailed construction of NetDAG. The construction of NetDAG is the greedy permutation construction framework of Har-Peled and

Algorithm 5: Procedure for updating the global Friends$(\cdot)$ and ServedBy$(\cdot)$

Input: new center p_k with radius r
1 let c'_k be the center that served p_k two phases earlier;
2 **foreach** $y \in$ Friends(c'_k) **do**
3 $\quad$ **if** $d(p_k, y) \le 4r$ **then**
4 $\quad\quad$ add y to Friends(p_k);
5 $\quad\quad$ add p_k to Friends(y);
6 **foreach** $y \in$ Friends(p_k) **do**
7 $\quad$ **foreach** $x \in$ ServedBy(y) with x not selected **do**
8 $\quad\quad d_{\text{new}} \leftarrow d(x, p_k)$;
9 $\quad\quad$ **if** $d_{\text{new}} < \delta(x)$ **then**
10 $\quad\quad\quad$ remove x from ServedBy(Center(x));
11 $\quad\quad\quad$ Center$(x) \leftarrow p_k$;
12 $\quad\quad\quad$ insert x into ServedBy(p_k);
13 $\quad\quad\quad \delta(x) \leftarrow d_{\text{new}}$;
14 $\quad\quad\quad$ update H with the new $\delta(x)$;

Mendel [25] with modifications. The greedy permutation iteratively generates the centers $p_0, p_1, \dots$ and so on. Each new center p_k is the farthest unselected point from all previously chosen centers, so, the radii $r_1 \ge r_2 \ge \cdots$ form a non-increasing sequence. The algorithm maintains two per-center structures: (i) the set Friends$(\cdot)$ (the *friends list*), and (ii) the set ServedBy$(\cdot)$. At the kth iteration, the friends list of a center p_i contains all centers whose distance to p_i is at most $\min\{8r_k, 4r_i\}$, where $4r_i$ reflects the radius when p_i was first selected and $8r_k$ is the current trimming threshold. ServedBy(p_i) stores the points for which p_i is currently the nearest selected center.

To describe Alg. 4, following [25], we define the term *phase*: A phase begins at iteration i and ends at the first $j > i$ with $r_{j-1} \le r_{i-1}/2$; that is, the radius halves once per phase.

- **Greedy permutation.** An arbitrary point p_0 is chosen as the first center. For every other point $x \in \mathcal{D}$, Alg. 4 computes $\delta(x) = d(x, p_0)$, where $\delta(x)$ always denotes the distance from x to its *current* nearest selected center. Each such point records its assigned center in Center(x), is inserted into ServedBy(p_0), and is pushed into the max-heap H keyed by $\delta(\cdot)$. Then, in every iteration, the algorithm extracts from H the farthest unselected point (Line 15), which becomes the next center p_k. $r_k = \delta(p_k)$ is exactly its distance to the nearest earlier center. When $k = 2$, this value determines the number of layers:
$$\phi = \lceil \log_2(r_2/\alpha) \rceil,$$
and the top layer is initialized as $Y_\phi = \{p_0\}$ (Line 19), which is an $(\alpha \cdot 2^\phi)$-net.
- **Layer formation.** Since the radii satisfy $r_1 \ge r_2 \ge \cdots$, whenever $r_k \le \alpha \cdot 2^j$ for the first time, the layer Y_j should be created. If r_k crosses multiple thresholds at once, all layers from j down to
$$i_{\min} = \max\left(1,\ \lceil \log_2(r_k/\alpha) \rceil\right)$$

Algorithm 6: Construction of EditPathTree

Input: G_{root}: root (center) graph; C: data graphs to be indexed
Output: $\mathcal{T}$: constructed EditPathTree

```
1 Initialize T with root G_root;
2 P ← ComputeGEPs(G_root, C);
3 Build a prefix tree T from all GEPs in P;
4 Compress T by removing single-child non-database nodes;
5 return T;
6 Function ComputeGEPs(G_root, C)
7     for G_i ∈ C do
8         P[G_i] ← the GEP from G_root to G_i;   // use AStar-BMao and
                canonical ordering
9     return P;
```

are created in sequence in Lines 23–28. Each layer Y_i is simply the prefix $\langle p_0, \ldots, p_{k-1} \rangle$, which, as a set $\{p_0, \ldots, p_{k-1}\}$, is an r_k-net by the greedy permutation [25].

- **Edge assignment.** For each newly created layer Y_i, the algorithm examines every $p_j^{i+1} \in Y_{i+1}$. Its potential children are restricted to Friends$(p_j^{i+1}) \cap Y_i$. For each such p_t^i that satisfies $d(p_t^i, p_j^{i+1}) \leq 3\alpha \cdot 2^i + 2\tau$, Alg. 4 adds an edge (p_j^{i+1}, p_t^i) (Line 28).
- **Updating friends lists and served points.** Line 30 of Alg. 4 invokes Alg. 5. This update procedure follows the friends-list maintenance strategy of Har-Peled and Mendel's net-tree construction [25]. Let c_k' be the center that served p_k two phases earlier (Line 1). Its friends list Friends(c_k') provides the complete set of candidate centers that may be inserted into Friends(p_k) (Line 2). For each candidate y, if $d(p_k, y) \leq 4r$, the pair is inserted into each other's friends list. Alg. 5 then examines all served points in ServedBy(y) for every $y \in$ Friends(p_k) (Lines 6–14). Whenever a point x satisfies $d(x, p_k) < \delta(x)$, the algorithm updates $\delta(x)$, reassigns x to ServedBy(p_k), and updates H accordingly. To control the size of friends lists, any center whose distance exceeds the $8r_k$ threshold is removed during scanning.
- **Termination.** The loop terminates once the radius reaches α; the prefix at that point is the α-net Y_0, which roots EPF, and Alg. 4 then connects each $p_j^1 \in Y_1$ to the Y_0 points within $3\alpha + 2\tau$ (edge set E').

A.2 Supplementary Analysis of EPF

This subsection establishes the space and query time complexity of EPF. Recall that non-database nodes store graphs produced along the GEP from the root to a data graph. We bound the size of individual non-database nodes (Lemmas A.11–A.12) and the total node count (Lemma A.13). To aggregate the per-node bounds into a whole-tree bound, we construct an injective mapping from non-database nodes to data nodes (Lemmas A.14–A.15), which yields the total size of all graphs (Lemmas A.16–A.17) and the total number of edit operations (Lemma A.18). These together give the space complexity of EPT (Corollary 3). Finally, we give an upper bound on the number of EPTs that should be traversed during a query (Lemma A.19).

Construction. The EPT is constructed in the following three steps. (1) **GEP computation.** For each data graph G_i, the GEP from the root G_{root} to G_i is extracted from the exact GED computation $d(G_{\text{root}}, G_i)$ by AStar-BMao [9] (Line 2): the edit operations induced by the returned vertex mapping form the GEP, namely a relabeling for each matched vertex pair with different labels, a deletion (resp. insertion) for each vertex of G_{root} (resp. G_i) left unmatched, and the corresponding edge edits. (2) **Prefix sharing.** GEPs that share a prefix of operations are merged in the prefix tree (Line 3). (3) **Path compression.** Single-child non-database nodes are compressed into single edges (Line 4). The construction algorithm is presented in Alg. 6.

First, we recall the terminology used in the analysis. Each node of an EPT stores a graph. A *data node* is a node whose graph (called a *data graph*) belongs to the graph database $\mathcal{D}$. A *non-database node* is a node whose graph does not belong to $\mathcal{D}$; such a graph (called a *non-database graph*) is produced along the GEP from G_{root} to some data graph.

Lemma A.10 (Edge-edit Bound Between Two Graphs). *Given two graphs G_1 and G_2, there exists an edit sequence from G_1 to G_2 whose* edge-edit operations *are at most $|E(G_1)| + |E(G_2)|$.*

By the definition of GEP in Sec. 2.1, a GEP is the shortest edit sequence transforming G_1 into G_2. There exists a trivial edit sequence that deletes all edges of G_1 and then inserts all edges of G_2, which performs exactly $|E(G_1)| + |E(G_2)|$ edge-edit operations.

Lemma A.11 (Upper Bound of the Size of Non-database Nodes in EPT). *Given an* EPT *rooted at G_{root} and a data graph set C, let $G \in C$ and let g be a non-database node that represents a non-database graph produced by the GEP from G_{root} to G. Then,*

$$|E(g)| \leq |E(G_{\text{root}})| + |E(G)|.$$

Proof. Since g appears as a non-database graph on the GEP from G_{root} to G, transforming G_{root} into g requires at least $|E(g)| - |E(G_{\text{root}})|$ edge insertions, and transforming g into G requires at least $|E(g)| - |E(G)|$ edge deletions. Thus the GEP performs at least $(|E(g)| - |E(G_{\text{root}})|) + (|E(g)| - |E(G)|) = 2|E(g)| - |E(G_{\text{root}})| - |E(G)|$.

By Lemma A.10, the GEP from G_{root} to G performs at most $|E(G_{\text{root}})| + |E(G)|$ operations. If $|E(g)|$ exceeded this amount, the above lower bound would contradict Lemma A.10. Therefore,

$$|E(g)| \leq |E(G_{\text{root}})| + |E(G)|.$$

□

Lemma A.12 (Upper Bound of the Vertex Count of Non-database Nodes in EPT). *Given an* EPT *rooted at G_{root} and a data graph set C, let $G \in C$ and let g be a non-database node that is a non-database graph produced by the GEP from G_{root} to G. Then*

$$|V(g)| \leq \max\{|V(G_{\text{root}})|,\ |V(G)|\}.$$

Proof. As shown in [8], a GEP between two graphs cannot contain both vertex insertion and vertex deletion operations. Hence, along the GEP from G_{root} to G, the vertex count changes monotonically. Therefore, every non-database graph on this GEP, including g, satisfies

$$|V(g)| \leq \max\{|V(G_{\text{root}})|,\ |V(G)|\}.$$

□

Lemma A.13 (EPF Node Count). *The total number of nodes in* EPF *is $O(N)$, where N is the number of graphs in the database.*

Proof. The total number of nodes in EPF equals the sum of the nodes across all its EPTs. Consider the i-th EPT that covers N_i data graphs, where K is the number of EPTs in EPF, $i = 1, \ldots, K$, and $\sum_i N_i = N$. Let m_i denote the number of leaf nodes in this

Algorithm 7: Construction of injective mapping π

Input: EPT with non-database node set I and data node set C
Output: Injective mapping $\pi : I \to C$

```
U ← C ;                                   // U: unassigned data nodes
foreach g ∈ I in order of decreasing depth do
    Let T_g be the subtree rooted at g;
    Choose any d ∈ T_g ∩ U;
    π(g) ← d;
    U ← U \ {d};
return π;
```

EPT; since every leaf corresponds to a data graph assigned to this EPT, we have $m_i \le N_i$. According to Def. 5.1, any non-database node must have multiple children. In a rooted tree, where every non-database internal node has out-degree at least 2, the number of such non-database nodes is bounded by the number of leaves, hence, at most m_i. Therefore, the total number of nodes in the i-th EPT is at most $N_i + m_i \le 2N_i$. Summing over all trees in EPF gives $\sum_i O(N_i) = O(\sum_i N_i) = O(N)$. □

Lemma A.14 (Number of Nodes in a Subtree of EPT). *Given an* EPT *with a data graph set C, consider any node u and the subtree $\mathcal{T}_u$ rooted at u. Let D_u be the set of nodes in $\mathcal{T}_u$ whose graphs are in C, and let I_u be the set of non-database internal nodes. Then*

$$|D_u| \;\ge\; |I_u|.$$

Proof. Consider an arbitrary node u and its subtree $\mathcal{T}_u$. By Def. 5.1 and the compression (Line 4 of Alg. 6), every non-database node in $\mathcal{T}_u$ has out-degree at least 2, and every leaf in $\mathcal{T}_u$ represents a data graph; let L_u denote the set of leaves in $\mathcal{T}_u$. Hence, $L_u \subseteq D_u$.

Similar to Lemma A.13, the number of non-database internal nodes in $\mathcal{T}_u$ is at most the number of leaves:

$$|I_u| \;\le\; |L_u|.$$

Since $L_u \subseteq D_u$, we obtain $|L_u| \le |D_u|$, and thus

$$|I_u| \;\le\; |L_u| \;\le\; |D_u|.$$

□

Lemma A.15 (Assignment of Data Nodes to Non-database Nodes). *Given an* EPT *with data graph set C, let I be the set of non-database nodes in this* EPT. *Then, there exists an injective mapping*

$$\pi : I \to C$$

such that $\pi(g)$ is in the subtree rooted at g for every $g \in I$.

Proof. We first give a construction of π in Alg. 7, and then show that it is injective and satisfies $\pi(g) \in \mathcal{T}_g \cap C$ for all $g \in I$.

Construction. We construct π as follows (Alg. 7). We process all non-database nodes in I in order of decreasing depth. When processing a non-database node g, we choose any data node in its subtree $\mathcal{T}_g$ that has not been assigned to any previously processed non-database node, and set $\pi(g)$ to be this data node.

Correctness. We show that Line 4 always finds an available data node for every non-database node g.

Let C_g and I_g be the sets of data nodes and non-database nodes in $\mathcal{T}_g$, respectively. By Lemma A.14, we have $|C_g| \ge |I_g|$.

When g is processed (Line 2), all previously processed non-database nodes in $\mathcal{T}_g$ are descendants of g (since we process by decreasing depth), so at most $|I_g| - 1$ of them have been assigned data nodes (Line 5) from C_g. Since $|C_g| \ge |I_g|$, at least one data node in C_g remains available for Line 4.

Since each data node is assigned at most once (Line 5), π is injective. By Line 4, $\pi(g) \in \mathcal{T}_g \cap C = C_g$ for all $g \in I$. □

Lemma A.16 (Total Size of Non-database Graphs in EPT). *Given an* EPT *rooted at G_{root} with data graph set C, let I be the set of non-database nodes in this* EPT. *Then, the total size of all non-data graphs satisfies*

$$\sum_{g\in I}\big(|V(g)| + |E(g)|\big) = O\big(|C| \cdot |E(G_{\text{root}})| + \sum_{G\in C} |E(G)|\big).$$

Proof. By Lemma A.15, there exists an injective mapping $\pi : I \to C$, such that for every $g \in I$, the data node $\pi(g)$ is in the subtree rooted at g.

Consider an arbitrary $g \in I$. Since $\pi(g)$ is in the subtree rooted at g, the path from G_{root} to $\pi(g)$ in EPT passes through g. Thus, g is a non-database graph on the GEP from G_{root} to $\pi(g)$. By Lemma A.11, $|E(g)| \le |E(G_{\text{root}})| + |E(\pi(g))|$. By Lemma A.12,

$$|V(g)| \le \max\{|V(G_{\text{root}})|,\ |V(\pi(g))|\} \le |V(G_{\text{root}})| + |V(\pi(g))|.$$

Therefore,

$$|V(g)|+|E(g)| \;\le\; \big(|V(G_{\text{root}})|+|E(G_{\text{root}})|\big)+\big(|V(\pi(g))|+|E(\pi(g))|\big).$$

Summing over all $g \in I$ gives

$$\sum_{g\in I}\big(|V(g)| + |E(g)|\big) \;\le\; |I| \cdot \big(|V(G_{\text{root}})| + |E(G_{\text{root}})|\big) + \sum_{g\in I}\big(|V(\pi(g))| + |E(\pi(g))|\big).$$

Since π is injective and $|I| \le |C|$,

$$\sum_{g\in I}\big(|V(g)| + |E(g)|\big) \;\le\; |C| \cdot \big(|V(G_{\text{root}})| + |E(G_{\text{root}})|\big) + \sum_{G\in C}\big(|V(G)| + |E(G)|\big).$$

Since $|V(G)| = O(|E(G)|)$ for every graph G,

$$\sum_{g\in I}\big(|V(g)| + |E(g)|\big) = O\big(|C| \cdot |E(G_{\text{root}})| + \sum_{G\in C} |E(G)|\big).$$

□

Lemma A.17 (Total Size of All Graphs in EPT). *Given an* EPT *rooted at G_{root} with data graph set C, let I be the set of non-database nodes in this* EPT. *Then, the total size of all graphs in* EPT *is*

$$O\big(|C| \cdot |E(G_{\text{root}})| + \sum_{G\in C} |E(G)|\big).$$

Proof. The total size of all graphs in EPT is

$$\sum_{g\in I}\big(|V(g)| + |E(g)|\big) + \sum_{G\in C}\big(|V(G)| + |E(G)|\big).$$

By Lemma A.16,

$$\sum_{g\in I}\big(|V(g)| + |E(g)|\big) = O\big(|C| \cdot |E(G_{\text{root}})| + \sum_{G\in C} |E(G)|\big).$$

Moreover, for each data graph $G \in C$ we have $|V(G)| = O(|E(G)|)$, and thus

$$\sum_{G\in C}\big(|V(G)| + |E(G)|\big) = O\Big(\sum_{G\in C} |E(G)|\Big).$$

Therefore, the total size of all graphs in EPT is

$$O\big(|C| \cdot |E(G_{\text{root}})| + \sum_{G\in C} |E(G)|\big).$$

□

Lemma A.18 (Total Number of Edit Operations on Edges in EPT). *Given an* EPT *rooted at G_{root} with a data graph set C, let $\mathcal{E}$ be*

the set of edges in this EPT*. For each edge* $e \in \mathcal{E}$*, let* $\mathrm{op}(e)$ *denote the number of edit operations represented by* e*. Then, the total number of edit operations represented by all edges in* EPT *satisfies the following*

$$\sum_{e\in\mathcal{E}} \mathrm{op}(e) = O\big(|C| \cdot |E(G_{\text{root}})| + \sum_{G\in C} |E(G)|\big).$$

Proof. For each data graph $G \in C$, the path from G_{root} to G in EPT represents a graph edit path (GEP) by Def. 5.1. Let $\mathrm{ops}(G)$ be the total number of edit operations represented by the edges on this path.

To derive the upper bound of $\mathrm{ops}(G)$, we consider the trivial edit sequence that transforms G_{root} into G by deleting all vertices and edges of G_{root} and then inserting all vertices and edges of G. This requires

$$|V(G_{\text{root}})| + |E(G_{\text{root}})| + |V(G)| + |E(G)| = O(|E(G_{\text{root}})| + |E(G)|)\,,$$

Since a GEP cannot be longer than this trivial sequence, we obtain

$$\mathrm{ops}(G) = O(|E(G_{\text{root}})| + |E(G)|)\,.$$

Each data graph $G \in C$ has a root-to-G path with $\mathrm{ops}(G)$ edit operations. The EPT is formed by merging shared prefixes of these paths into a tree. So, each tree edge appears on one or more such paths. Therefore, the total number of edit operations in the tree is at most the sum over all paths:

$$\begin{aligned}\sum_{e\in\mathcal{E}} \mathrm{op}(e) &\le \sum_{G\in C} \mathrm{ops}(G)\\ &= O\Big(\sum_{G\in C} \big(|E(G_{\text{root}})| + |E(G)|\big)\Big)\\ &= O\big(|C| \cdot |E(G_{\text{root}})| + \sum_{G\in C} |E(G)|\big).\end{aligned}$$

□

Corollary 3 (Space Complexity of EPT)**.** Given an EPT rooted at G_{root} with data graph set C, let I be the set of non-database nodes and $\mathcal{E}$ be the set of edges, both in this EPT. Then the total space of EPT, including all graphs stored at nodes and all edit-operation sequences represented by its edges, is

$$O\big(|C| \cdot |E(G_{\text{root}})| + \sum_{G\in C} |E(G)|\big).$$

Proof. By Lemma A.17, the total size of all graphs stored at the nodes in EPT is

$$O\big(|C| \cdot |E(G_{\text{root}})| + \sum_{G\in C} |E(G)|\big),$$

and by Lemma A.18, the total number of edit operations represented by all edges in EPT is:

$$O\big(|C| \cdot |E(G_{\text{root}})| + \sum_{G\in C} |E(G)|\big).$$

Therefore, the total space of EPT is:

$$O\big(|C| \cdot |E(G_{\text{root}})| + \sum_{G\in C} |E(G)|\big).$$

□

Finally, we give an upper bound on the number of EPTs traversed during a query.

Lemma A.19 (Upper bound on the number of EPTs to traverse). *Consider a graph similarity search query* (Q, τ) *in* Gisma*, where each* EPT *is built for a ball* $B(p, \alpha)$ *of radius* α *centered at a leaf pivot* p *(thus the set of* EPT *roots forms an* α*-net). Then, the number of* EPT*s that may contain answers (hence need to be traversed) is at most*

$$\lambda_\alpha^{\log_2\left(4(1+\tau/\alpha)\right)}.$$

Proof. Fix a query (Q, τ). By construction, an EPT rooted at p indexes the data graphs inside $B(p, \alpha)$. If this EPT contains at least one answer, then there exists a graph g with $d(g, Q) \le \tau$ and $d(g, p) \le \alpha$. By the triangle inequality,

$$d(Q, p) \;\le\; d(Q, g) + d(g, p) \;\le\; \tau + \alpha \;=\; \alpha + \tau.$$

Hence, any EPT that contains an answer must have its root within the ball $B(Q, \alpha + \tau)$.

At the leaf layer, the roots of EPTs form an α-net; in particular, they are α-separated. Therefore, the set of candidate roots $Y' = \{\, p : \; d(Q, p) \le \alpha + \tau \,\}$ is an α-separated subset contained in a ball of radius $R = \alpha + \tau$. Applying Corollary 1 with $r = \alpha$ and $R = \alpha + \tau$ yields

$$|Y'| \;\le\; \lambda_\alpha^{\log_2\left(\frac{4R}{r}\right)} \;=\; \lambda_\alpha^{\log_2\left(4(1+\tau/\alpha)\right)}.$$

Hence, at most $\lambda_\alpha^{\log_2\left(4(1+\tau/\alpha)\right)}$ EPTs need to be traversed. □

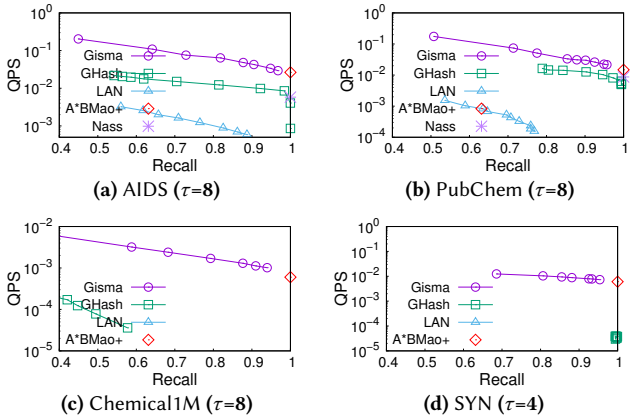
Query time complexity. As analyzed in Sec. 5.1, each EPT is queried via a DFS traversal visiting each node at most once (Alg. 2), and the number of nodes in a EPT rooted at p is $O(|C_p|)$ by Lemma A.13. By Lemma A.19, the number of EPTs that need to be traversed is at most $\lambda_\alpha^{\log_2(4(1+\tau/\alpha))}$. Therefore, the total query time of EPF is $O(\sum_j |C_{p_j}|)$, where the sum is over the searched EPTs. At each visited node, the algorithm performs at most one GED computation, so the worst-case NDC of a single EPT is $O(|C_p|)$, the same as a linear scan of the data graphs it indexes. The three optimizations do not increase this bound: subtree pruning (Sec. 5.1.1) checks the precomputed $\bar{d}_G$ in $O(1)$ per node, lower-bound propagation (Sec. 5.1.2) computes lower bounds using the precomputed $d(G_p, G_c)$ in $O(1)$ per node, and search tree reuse (Sec. 5.1.3) leaves the NDC unchanged while reducing the per-computation cost.

A.3 Supplementary Experimental Results

Supplementary figure for EXP-2. Fig. 10 presents the QPS-recall curves with a logarithmic QPS axis, complementing Fig. 8 in the main text. The log scale better reveals the relative performance differences among methods with vastly different QPS values, such as exact methods (AStar-BMao+, Nass) and approximate methods.

Effectiveness of search tree reuse. We conduct a study on the effectiveness of search tree reuse presented in Sec. 5.1.3. *With Reuse* measures the average running time of single-pair GED computations with search tree reuse during the query processing of Gisma under the default setting. *Baseline* measures the average running time of the original App-BMao+ without reuse on the same graph pairs for which reuse is triggered. Our reuse strategy achieves speedups of 1.54× to 3.41× (AIDS: 2.97×, PubChem: 3.41×, Chemical1M: 3.06×, SYN: 1.54×), reducing average runtime from 12.90ms to 4.00ms on all datasets.

Scalability. We evaluate the scalability of Gisma by varying the database size N on the SYN dataset with $\tau = 4$. Fig. 11 shows the



(a) AIDS (τ=8) (b) PubChem (τ=8)

(c) Chemical1M (τ=8) (d) SYN (τ=4)

Figure 10: QPS-recall curves (log scale)

runtime as N increases from 10K to 1M. The runtime grows linearly with N, confirming the scalability of Gisma.

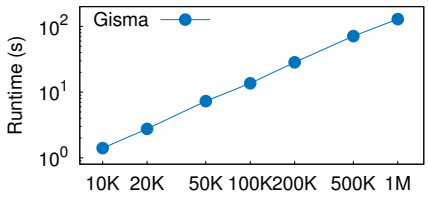


Figure 11: Scalability of Gisma **on** SYN **(τ = 4), with database size N varying from 10K to 1M.**

Full ablation study. Tab. 10 reports the ablation study of Gisma on all four datasets, complementing Tab. 8 of the main text, which reports AIDS and SYN.

A.4 Characterizing and Verifying the Two-Stage Curve

Diversity of the real datasets. Tab. 11 reports the statistics of the six real datasets on which we plot $\max_p \lambda(p, r)$ (Figs. 4(a), 5, and 12). They span a wide range of graph statistics, so the two-stage shape is not tied to any particular one: the average graph size ranges from 13 vertices (the social-network IMDB) to 48 (PubChem); the density ranges from 0.05 (PubChem) to 0.77 (the dense IMDB ego-networks), with the sparse molecular graphs at 0.09–0.11; and the label distribution ranges from a single label (the unlabeled IMDB) to 94 labels (Chemical), with the most frequent label covering from 43–44% of the vertices on the more uniform PubChem and Mutagenicity to about 73% on the carbon-dominated AIDS, Chemical, and CANCER. The sharp-peak/long-tail shape appears on all of them (Fig. 12).

Two-stage fit. We characterize the two stages with a per-stage fit rather than a single closed form for the whole curve, so that the fit reflects the two-stage phenomenon instead of blurring it into one smooth shape. We split each curve at the threshold radius α, the actual boundary between the two stages (NetDAG handles GED $\geq \alpha$ and EPF handles GED $< \alpha$). The rising stage ($r \leq \alpha$) is fitted by a skew-normal density $\max_p \lambda = A\,\mathrm{sn}(r;\, \xi, \omega, a)$, whose skewness a

Table 10: Full ablation study of Gisma **on all four datasets (runtime in seconds). Tab. 8 of the main text reports** AIDS **and** SYN**.**

Dataset	Method	Runtime (s)					
AIDS		τ=2	4	6	8	10	12
	Gisma	0.0070	0.1041	**1.405**	**20.41**	**259.9**	**2291**
	Gisma-no-GS	0.0062	0.1062	1.445	21.30	264.2	2357
	Gisma-no-SS	0.0059	0.1048	1.599	23.38	276.3	3059
	App-BMao+	**0.0046**	**0.1019**	1.752	25.04	278.2	3120
	Gisma-no-Reuse	0.0075	0.1314	1.856	23.45	297.9	2625
	Gisma-no-SP	0.0088	0.1228	1.634	22.95	291.9	2588
	Gisma-no-LP	0.0069	0.0931	1.506	21.68	287.7	2498
PubChem		τ=2	4	6	8	10	12
	Gisma	**0.0214**	**0.2025**	**2.445**	**30.48**	**323.6**	**3093**
	Gisma-no-GS	0.0234	0.2398	3.217	34.74	348.4	3205
	Gisma-no-SS	0.0221	0.2433	3.672	38.69	404.5	4664
	App-BMao+	0.0238	0.2511	3.853	44.82	426.9	4822
	Gisma-no-Reuse	0.0189	0.2961	3.599	36.46	346.0	3221
	Gisma-no-SP	0.0257	0.2425	2.919	35.14	333.9	3186
	Gisma-no-LP	0.0246	0.1968	2.864	33.00	331.3	3166
Chemical1M		τ=2	4	6	8	10	–
	Gisma	0.1920	**4.018**	**66.87**	**801.3**	**8247**	–
	Gisma-no-GS	0.1944	4.372	72.76	837.2	8765	–
	Gisma-no-SS	0.2052	4.559	80.24	950.1	10874	–
	App-BMao+	**0.1715**	5.225	87.76	956.7	11314	–
	Gisma-no-Reuse	0.2892	4.301	75.87	962.7	9329	–
	Gisma-no-SP	0.2077	4.010	73.66	932.4	9097	–
	Gisma-no-LP	0.2184	3.902	73.11	872.1	8778	–
SYN		τ=1	2	3	4	5	6
	Gisma	**0.8149**	**7.237**	**35.15**	**129.2**	**455.2**	**1259**
	Gisma-no-GS	0.8179	7.271	37.66	130.6	485.8	1311
	Gisma-no-SS	0.8162	7.254	38.67	140.3	527.4	1380
	App-BMao+	0.8201	7.305	40.90	142.8	552.3	1416
	Gisma-no-Reuse	0.8119	8.591	37.68	145.5	513.5	1403
	Gisma-no-SP	0.9334	8.348	36.77	143.7	507.3	1396
	Gisma-no-LP	0.8866	7.446	35.89	143.9	502.4	1381

Bold indicates the best (lowest) runtime; underline indicates the second-best one, among Gisma, Gisma-no-GS, Gisma-no-SS, and App-BMao+.

Table 11: Statistics of the six real datasets used for the two-stage curve, showing their diversity in size, density, and label distribution. Density is the average of $2|E|/(|V|(|V|-1))$ over graphs; top-k is the share of vertices covered by the k most frequent labels. Statistics for Chemical1M **and CANCER are computed on a representative sample.**

Dataset	$\lvert\mathcal{D}\rvert$	Avg. $\lvert V\rvert$	Density	#label	top-1	top-3	top-5
AIDS	42.7K	25.6	0.10	62	72.5%	95.7%	98.5%
PubChem	22.8K	48.1	0.05	10	43.1%	91.8%	98.4%
Chemical1M	1M	24.0	0.11	94	74.3%	95.8%	98.5%
IMDB	1.5K	13.0	0.77	1	100%	100%	100%
Mutagenicity	4.3K	30.3	0.09	14	44.3%	93.3%	98.9%
CANCER	32.6K	26.3	0.10	59	72.8%	96.1%	98.6%

accommodates the asymmetric rise and peak; the tail stage ($r \geq \alpha$) is fitted by an exponential decay $\max_p \lambda = G(\alpha)\, e^{-k(r-\alpha)}$, anchored at the rising stage's value $G(\alpha)$ so the two stages meet continuously at α. We fit the six real datasets: the molecular AIDS, PubChem, and Chemical1M, which we search in our experiments, are split at the same $\alpha = 12$ used there; the additional datasets IMDB, Mutagenicity, and CANCER, included only to illustrate that the shape holds across domains and not used for search, are split at the first radius past the peak where the curve falls below half the peak height (α = 28, 21, and 11, respectively). The fit is close on all six datasets: the skew-normal rising stage attains R^2 from 0.87 to 1.00, and the exponential tail from 0.88 to 1.00, with $p < 0.001$ for every fitted stage (Fig. 12). The peak height varies with dataset size and label diversity, ranging

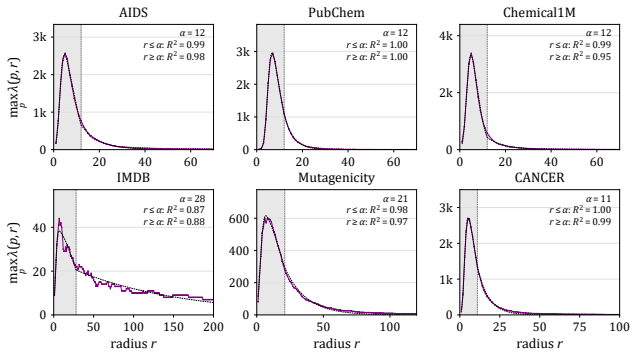


Figure 12: Two-stage fit for the six real datasets, on linear axes. Each curve is split at the threshold radius α (grey dashed line, with the shaded band marking $r \le \alpha$) into a rising stage fitted by a skew-normal density and a tail stage fitted by an exponential decay $G(\alpha)\, e^{-k(r-\alpha)}$ (both drawn as black dashed lines), which meet continuously at α; each panel reports the R^2 of the two stages, both fitted with $p < 0.001$.

from 44 for the small, unlabeled social graphs of IMDB to 2,692 for CANCER.

Synthetic-grid robustness. To test the two-stage phenomenon under controlled variations of graph size, density, and label distribution, we build a grid of 18 synthetic datasets that vary the node count ($\in \{10, 13, 16\}$), the edge density ($\in \{0.20, 0.25, 0.30\}$), and the number of node labels ($\in \{2, 4\}$), together with the benchmark SYN dataset as the single-label case; each dataset contains 100K graphs generated with the GraphGen generator and is embedded with a per-dataset GREED model trained on exact GED. The node count is capped at 16 because exact GED becomes intractable for larger i.i.d. random graphs. Fig. 13 plots the resulting $\max_p \lambda(p, r)$ curves. Every one of these synthetic datasets exhibits the same sharp-peak/long-tail shape as the real datasets, and the peak shifts to a larger radius as the graph size grows (as expected, since larger graphs have larger GEDs), with the peak height ranging from 2,161 to 8,069. This confirms that the two-stage phenomenon is a structural property of the GED space rather than an artifact of any particular dataset.

A.5 Compression of the EditPathForest

Tab. 12 gives the per-dataset breakdown of the EPF compression reported in Sec. 5, separating its two sources. The total compression is the product of prefix sharing and the average number of edit operations packed onto each edge, and the edge percentage is its reciprocal.

A.6 Index Construction Time

Tab. 6 (main text) reports the total index construction time. Gisma and LAN each build a *single* index that answers queries for all τ, whereas Nass builds a *separate* index per τ; the main-text Nass entry is its build time at the default τ. Tab. 13 gives Nass's full per-τ breakdown (wall-clock, 200 threads, 2 GB budget): its cost grows sharply with τ and becomes infeasible (CNB: cannot be built within one week) at large τ and on the large datasets. LAN is built with its GED-based distance on AIDS and PubChem, and with the same GREED embedding distance as Gisma on SYN and Chemical1M, where GED-based construction is too expensive.

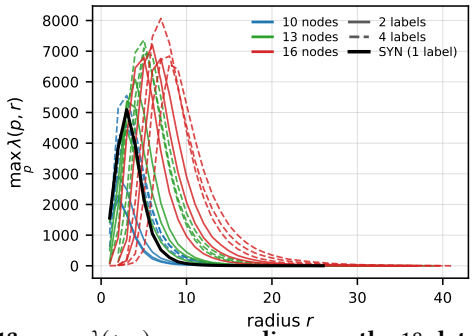


Figure 13: $\max_p \lambda(p, r)$ versus radius r on the 18-dataset synthetic grid (coloured by node count; solid: 2 labels, dashed: 4 labels) together with the benchmark SYN dataset (the single-label case; thick black). Every curve shows the same two-stage shape, and the peak shifts to a larger radius as the graph size grows.

Table 12: EPF compression at the default configuration (τ=4 for SYN, τ=8 otherwise), measured over the data graphs stored as EPF leaves. Each entry $a \times b \approx c$ reports the prefix sharing a, the average edit operations per edge b, and their product, the total compression c; the edge percentage is the reciprocal of c.

Dataset	Compression (prefix × ops/edge)	Edge %
AIDS	$1.46 \times 4.89 \approx 7.13\times$	14.0%
PubChem	$1.62 \times 5.39 \approx 8.72\times$	11.5%
Chemical1M	$2.18 \times 3.75 \approx 8.17\times$	12.2%
SYN	$1.54 \times 3.52 \approx 5.42\times$	18.4%

Table 13: Nass index construction time per threshold τ (CNB: cannot be built within one week).

τ	AIDS	PubChem	Chemical1M
2	55 s	25 s	CNB
4	3.7 min	3.2 min	CNB
6	1.73 h	54 min	CNB
8	39.0 h	21.6 h	CNB
10	CNB	CNB	CNB

τ (SYN)	1	2	3	4	5
time	1.53 h	17.9 h	CNB	CNB	CNB

A.7 Search-time Memory

We report Gisma's search-time working memory and traversal cost on all four datasets, at the τ of the main comparison (Tab. 14). The working memory is 158 MB on AIDS and 188 MB on PubChem, and rises to 1927 MB on SYN and 2468 MB on the million-graph Chemical1M. Per query, the search enters only a subset of the EPTs (19–62%) and locates answers at a shallow depth. As τ increases from 2 to 12 on AIDS, the working memory grows from 65 to 259 MB and the fraction of EPTs entered from 11% to 27%, while the average answer depth stays shallow (9.6–10.6).

A.8 Applications Requiring Large τs

Production ligand discovery permits large GEDs. The publicly available SmallWorld interface at `sw.docking.org` [41], a GED search engine of a multi-billion-molecule ligand discovery

Table 14: Gisma's search-time working memory and traversal cost. "EPT entered": the fraction of EPTs the query descends into; "depth": the average depth at which answers are found.

Dataset	τ	Mem (MB)	EPT entered	Depth
AIDS	2	65	366/3399 (11%)	9.6
	4	78	497/3399 (15%)	9.6
	6	99	626/3399 (18%)	10.0
	8	158	745/3399 (22%)	10.3
	10	217	851/3399 (25%)	10.5
	12	259	927/3399 (27%)	10.6
PubChem	8	188	997/2459 (41%)	8.2
SYN	4	1927	10610/17078 (62%)	5.2
Chemical1M	8	2468	2306/12372 (19%)	9.9

Table 15: Pairwise GED among the 30 ChEMBL actives with the highest pChEMBL values in each activity class of Hert et al. [26]. All GEDs are exact.

Activity class	ChEMBL target	Pairs	GED > 8
5-HT3 antagonists	CHEMBL1899	406	89.7%
5-HT1A agonists	CHEMBL214	435	79.1%
5-HT reuptake inhibitors	CHEMBL228	435	77.5%
D2 antagonists	CHEMBL217	435	95.2%
Renin inhibitors	CHEMBL286	406	87.7%
Angiotensin II AT1 antagonists	CHEMBL227	435	80.5%
Thrombin inhibitors	CHEMBL204	435	91.3%
Substance P (NK1) antagonists	CHEMBL249	435	94.3%
HIV protease inhibitors	CHEMBL243	406	91.4%
Cyclooxygenase inhibitors	CHEMBL230	435	62.5%
Protein kinase C inhibitors	CHEMBL299	435	77.0%
Overall		**4,698**	**84.1%**

service [48], permits distance queries up to 16. It also reports that distant matches are the bottleneck of the search: "*SmallWorld search time is nonlinear: close neighbors are found almost instantly, but more distant matches take much longer.*"

Nearest neighbours of molecules are at large GEDs. On the Briem–Lessel benchmark, searching for the ten nearest neighbours of a drug-sized query molecule reaches a median distance of 8 edits, with individual searches ranging from 2 to 24 edits [47]. That is, even the nearest neighbours of a molecule are often not within a small GED.

GEDs among molecules of the same biological activity. We examine how large the GED is among molecules that share a biological activity. Hert et al. [26] group molecules into 11 classes by their protein targets, and ChEMBL [39] groups molecules by the same criterion. For each of the 11 classes, we take from ChEMBL the 30 actives with the highest pChEMBL values [5] (pChEMBL ≥ 6, *i.e.,* IC50/Ki $\leq 1\mu$M) and compute their pairwise GED. Across the 4,698 same-target pairs, 84.1% have a GED larger than 8, and the fraction exceeds 60% in every class (Tab. 15). That is, molecules of the same biological activity are often not within a small GED of each other.

A.9 Sensitivity to the GED Verifier

Gisma's verifier can be replaced by another one. To show that the two-stage index does not depend on a particular verifier, we replace Gisma's verifier with one based on LSa [8]. As A*-LSa [8] computes the exact GED, we derive its approximate counterpart, denoted App-LSa, by limiting the number of search states it visits, following App-BMao [51]. We compare Gisma, which uses App-LSa as its verifier, with App-LSa+ and A*-LSa+, which verify every graph in the database with App-LSa and A*-LSa, respectively. Gisma and

Table 16: Gisma with the LSa lower bound against the full-scan baselines using the same lower bound, at the default τ. Format: seconds per query (recall%). Bold: the lowest runtime. CNF: cannot finish in 24h.

Dataset	Gisma	App-LSa+	A*-LSa+
AIDS (τ=8)	**27.48** (88.69)	28.31 (90.10)	74.44 (100)
PubChem (τ=8)	**34.27** (88.43)	39.62 (91.21)	113.16 (100)
SYN (τ=4)	**1168.8** (91.48)	1169.0 (90.67)	CNF
Chemical1M (τ=8)	**859.8** (89.01)	990.4 (89.09)	CNF

Table 17: Gisma's update cost. Insertion "compute" is the single-thread work per graph; "batch" is the batched parallel wall-clock per graph; deletion masks in $O(1)$. Recall cells show % before / after the operation on 10% of the graphs; precision stays 100%.

Dataset	Ins. compute (s/g)	Ins. batch (s/g)	Del. (μs/g)	Ins. recall	Del. recall
AIDS	2.40	0.056	1.50	90.61/90.51	90.51/90.31
PubChem	5.81	0.122	1.35	90.61/91.36	90.77/91.07
SYN	0.11	0.0019	1.83	91.06/90.49	90.51/90.34
Chemical1M	4.23	0.044	1.76	91.54/91.54	91.58/91.61

App-LSa+ use the same limit on the number of search states, under which the recall of both is around 90%. The results are reported at the default τ (τ=4 for SYN, τ=8 otherwise) in Tab. 16. Gisma is the fastest on all four datasets at a comparable recall, and A*-LSa+ cannot finish on SYN and Chemical1M.

A.10 Update Cost

Gisma can be updated by insertion and deletion (Sec. 5). We validate each operation on 10% of the graphs per dataset (4258 on AIDS, 2269 on PubChem, and 99,990 on SYN and Chemical1M); precision stays at 100% and recall changes by at most 1% in every case (Tab. 17). To insert a graph, NetDAG locates its nearest EPT root, the edit path to that root is computed, and the graph is merged into the EPT; insertions are batched and run in parallel, costing 0.0019–0.122 s per graph. Deletion masks a graph in $O(1)$ (1.3–1.8 μs per graph).